Comparative Analysis of RSA and ECC in a Blockchain-Based E-Voting System

A Project Report Submitted
in Partial Fulfilment of the Requirement
for the Degree in Bachelor of Science (Computer Science)

By

Bidipta Das

S2201399

22003056



Department of Computer Science
St. Edmund's College
Shillong - 793003

June,2025

Department of Computer Science

St. Edmund's College

Shillong - 793003

CERTIFICATE

This is to certify that the project titled "Comparative Analysis of RSA and ECC in a Blockchain-Based E-Voting System" is Bonafide work done by Bidipta Das bearing university roll number S2201399, under my guidance during the 6th Semester in Partial Fulfilment of the Requirement for the Degree in Bachelor of Science (Computer Science)

Prof. Dr. Sangyal Lama. Tamang

Internal Guide

The project seminar was held on 6th June 2025 at St. Edmund's College, Shillong.

Ms. Preeti Thapa,

H.O.D.,

Dett. of Computer Science,

St. Edmund's College, Shillong.

Table of Contents

Sl No.	Content	Page No.
01	Acknowledgement	4
02	Abstract	5
03	Introduction	6-14
04	Synopsis	15
05	Feasibility study	16
06	Analysis	17-20
07	Hardware and Software Requirements	21
08	Design	22-29
09	Future Study	30
10	Screenshots of Developed System	31-33
11	Conclusion	34
12	Code	35-85
13	List of Figures	86
14	Abbreviation	87
15	Bibliography	88

Acknowledgement

I am very glad to complete this project. I could never image that I could learn so much from this project. It helped me enhance my knowledge and experience. There are many who have helped me complete this project and I would like to thank them.

My parents for always supporting me through thick and thin. My family for always understanding my dedication, effort and time I have put into this project.

My teachers with an unwavering effort and support always took into account of the smooth running of my project. They helped me and guide me throughout the project and consistently encouraged me to do more better in this project.

I want to specially thank Sir Sangyal L. Tamang, who has shown me the effective way of carrying out the project in the given time frame. His consistent encouragement and dedication helped me reach this milestone.

Last, but certainly not the least, I want to thank St Edmunds' College for providing me with this opportunity which will undoubtedly play a significant role in advancing my career.

Abstract

Election plays a very important role in every democratic country. Security and transparency are the essential concerns of the voters during an election. Blockchain technology is an immutable, encrypted, distributed ledger technology. A brief study on the encryption algorithms have been done in this project. Encryption algorithm ensures secure transaction, ensure data integrity and enable secure decentralized system. Exclusively a vivid comparison has been done on Rivest Shamir Adleman (RSA) and Elliptic Curve Cryptography (ECC) encryption algorithm. A plotted graph is used to analyze the performance between the two algorithms.

In this project the encryption algorithms are implemented in a blockchain-based e-voting system. This project has mainly focused in voter authentication, digital signature of the voter, building up the blockchain, encryption of the vote and decryption of the vote.

Introduction

Traditional paper-based voting systems raise several concerns about reliability, transparency, and cost. Some of the issues are:[6]

- Trustworthiness of notaries and the potential tampering of votes before counting.
- Total number of voter turnout.
- High costs associated with running elections—materials, logistics, and staff—especially when scaled to thousands of polling centers and repeated multiple times a year.

The e-voting process to be implemented in a large-scale scenario, requires the addressing of the above issues.

In digital system it is easy to recuse or copy the digital tokens (i.e., spend the same token twice). The receiver needs a reference that the token they are using hasn't been used earlier and also the token won't be invalid in conflicting transaction later.

In a centralized system, a trusted third party tracks all the transaction. They ensure no token is spent twice. All transactions are visible to them and they can enforce a single timeline [2].

Blockchain comes up as a solution to the problem for the following reason.

- Decentralized Network-Such a network consists of multiple interconnected nodes that can route data independently. This architecture improves redundancy and resilience, as the failure of one or more nodes doesn't compromise the entire system. The most redundant topology is fully decentralized, where each node can communicate directly with any other node[11]
- Public Broadcasting Transaction-All transactions are announced to the entire network. Every participant (nodes) receives and stores them. This ensures no secret or hidden transaction exist.
- Consensus on every transaction- Consensus in blockchain refers to the process by which all decentralized nodes in the network agree on the validity of data to be added to the blockchain. This ensures that only universally accepted blocks are appended, maintaining the integrity and consistency of the distributed ledger. Consensus is achieved by algorithms like PoW particularly used in cryptocurrency blockchain like Bitcoin [11].
- Immutable nature of the block- Once a transaction is added to a block and enough blocks are added after it, it becomes computationally impossible to reverse. Once

- blocks are added into the blockchain they can neither be removed or modified. As bocks are added they become a permanent part of the blockchain i.e., conceptualized

  as adding “links ” as chains which grows with each addition. Each block contains a timestamp in addition to a hash value of the header of the previous block which links the data in a chain of block. This gives the receiver the confidence that the transaction is valid and final [2][11].
- Encryption- Blockchain highly depends on Public key encryption where the public key is shared with everyone and the private key is kept secret. Every participant in the network has their own unique key pair. For example, if Charlie wants to send a secure message to Leah, he can use Leah’s public key to encrypt the message. Only Leah’s private key can decrypt this message, making it inaccessible to others—even if they intercept it. Another essential cryptographic element in blockchain is the hash function. Hash functions take an input of any size and return a fixed-size output known as a hash value. These are one-way functions, meaning it's virtually impossible to reverse the process and retrieve the original input from the hash. However, the same input will always generate the same hash, ensuring data integrity. Hashes are crucial in blockchain for linking blocks securely and for verifying data has not been tampered with [11].

Blockchain being a technology which uses distributed ledgers can be ideally used for this process The foundation of blockchain was first laid in 1991[1] in which the author talks about the authentication of digital documents using hash functions which can be accomplished through a central authority or timestamping hash functions. The term blockchain was first coined in 2008 in the paper Bitcoin Cryptocurrency. This paper proposed a system for electronic transaction without relying on trust. It proposes peer-to-peer network using proof of work to maintain public, temper-resistant transaction history [2].

Blockchain is a relatively new technology but its uses have exponentially expanded in every field. It is a decentralized and distributed data structure which records in chronological order a specific topology of data called transaction. This transaction is maintained by a network of computer worldwide. Unlike traditional system which involves a central authority, blockchain is managed by

community making it tamper-proof and transparent. Every transaction is verified by network algorithm and after being verified the transaction is added to the chain of previous transaction forming a secure and tamper-proof blockchain. Its distributed nature keeps the transaction transparent and ensures that no single entity can control or manipulate the data.[3]

Blockchain are classified into three types: public blockchain and private blockchain.[5][6]
Public blockchain- It is a decentralized, permissionless network where any one can participate, view and validate transaction. It is open to public and no central authority controls it. It is cost effective, slower in speed and highly scalable. Public blockchain are run in platform like Ethereum
Private blockchain- It is a permissioned centralized network where only authorized participants can access, validate and manage transaction. It is typically managed by a single organization or consortium. More expensive, Slower in speed and less scalable. Private blockchain are run in platform like Hyperledger fabric and Corda.
Consortium or hybrid blockchain- This combines the aspects of both public and private blockchain to allow selective decentralized. Jointly governed by a group of institution, balancing decentralization and access control.[10][11]

In sectors like healthcare, where regulations such as HIPAA and HITECH govern data privacy, permissioned blockchains are preferred to ensure compliance and data security. Nonetheless, advancements in encryption technologies have enabled experimental implementations of permissionless blockchains for electronic medical records (EMRs). These solutions suggest that as both blockchain technology and regulatory frameworks evolve, even broader applications may become feasible in the future.[11]

According to Melanie Swan Blockchain has evolved through two phases: -
Blockchain 1.0-The era to digital currency

Mainly focused on digital currency and peer-to-peer payment systems. It involves integration of multiple technologies for decentralized transaction. Bitcoin is the representative platform.

Blockchain 2.0-The era of smart contracts

Focus beyond digital currency-support programmable logic and asset transfer. Focuses in developing and deploying smart contracts. Ethereum is the representative platform.

The blockchain architecture consist of the following layers:

- Network Layer-Handle data propagation and verification mechanism across nodes.
- Consensus Layer-Implements mechanisms that allow distributed nodes to agree on the validity of blocks in a trustless, decentralized environment.
- Data Layer-This layer primarily utilizes a block-based data structure to ensure the integrity and immutability of stored information.
- Contract Layer-This layer supports the programmable capabilities of blockchain systems through the use of smart contracts.
- Application Layer-includes the end-user interfaces and practical implementations of blockchain platforms such as Bitcoin, Ethereum and Hyperledger**.**

Cryptography is central to blockchain's security, affecting all its layers-from data and network to consensus and application layers. It is essential for protecting user privacy, securing transaction information, maintaining data consistency and enforcing system integrity.[10]

The core role of cryptography in the blockchain technology in addressing the security challenges in blockchain.
Firstly, it simplifies and explains the underlying infrastructure of blockchain, providing a foundational understanding of its layered architecture
Secondly, it introduces cryptographic techniques—such as hash functions, asymmetric encryption, and digital signatures—to demonstrate how cryptography supports blockchain functionality.
Finally, it examines existing security issues in blockchain, including data integrity risks, consensus vulnerabilities, and privacy concerns [10].

Cryptographic algorithms are classified into three types based on the number of keys used in encryption and decryption. They are:

1.Secret Key Cryptography (SKC)-This method uses only a single key for encryption and decryption. SKC schemes can be divide into block cipher and stream cipher [4].

- Stream cipher- encrypt data bit-by-bit and use a feedback mechanism that constantly changes the key.
- Block cipher- encrypt data in fixed-size block using the same key for each block.

2.Public Key cryptography (PKC)-This scheme uses two keys; one for encryption (public key) and another for decryption (private key). This enables parties to share secret information securely over insecure network. While the public key is openly shared, the private key remains hidden.
RSA was the first PKC and still very widely used for key exchange and digital signature. The major advantage of PKC is simplified key management. Additionally, many PKC provide efficient digital mechanism for verifying authentication.
3.Hash Function (HF): This passage explains Hash Functions (HF), which use a mathematical transformation to generate a fixed-length hash value from plaintext. Unlike encryption algorithms, hash functions do not use keys and are irreversible, meaning the original data cannot be recovered from the hash. They are mainly used to create a digital fingerprint of data, helping to verify integrity and detect tampering. Hash functions are commonly used in file verification and password protection by operating systems.

The two main types of cryptography as the process of converting plaintext into ciphertext to protect data from unauthorized access are Symmetric Key Encryption and Asymmetric Key Encryption.
Symmetric algorithms like AES, BLOWFISH, DES, and 3DES are ideal for applications such as wireless communication, JFile handling, image processing, smart cards, and e-commerce services. Whereas Asymmetric algorithms like RSA, DSA, and ECC are best suited for internet banking, web applications, email verification, key exchange, and mobile platforms [8].
Symmetric Encryption: Uses the same key for both encryption and decryption. Its limitation is Key distribution is difficult over unsecured channels.
Asymmetric Encryption (Public Key Encryption):Uses a public key (for encryption) and a private key (for decryption). This allows secure communication over unsecured networks. The drawbacks of this asymmetric encryption is slower performance and relatively weaker encryption strength. Security of this encryption mechanism depends on the difficulty of certain mathematical problems (e.g., factorization, discrete logarithm).

A comparative analysis of cryptographic algorithms in 2019[8] reveals that AES (Advanced Encryption System) remains the most secure and widely used encryption method today, particularly in sensitive environment like government, banking and wireless communication. It offers exceptional strength especially in its 256-bit form, and currently has no practical attack against it.

Blowfish is known for its speed and efficiency in 32-bit system and is well suited for application where keys do not change frequently such as in a database platform and an e-commerce platform. On the other hand, DES and its higher version 3DES are older standards-DES offer fast transmission with low security, making it suitable for image transmission, while 3DES despite being slow in software is still used in smart cards and hardware-based system due to its cost effectiveness.

For Asymmetric encryption RSA is known for its strong security and is widely used in mobile banking and secure transmission, though it suffers from slow processing and high computational overhead due to large key size. DSA(Digital Signature Algorithm) is optimal for tasks like email and web authentication, offering good efficiency and security for signature verification. ECC(Elliptical Curve Cryptography) with its small key size and high efficiency, is becoming the preferred choice for secure communication in web and mobile environments. It provides strong encryption with faster performance and lower resource requirements, making it ideal for modern, resource-constrained devices. Overall, the choice of algorithm depends on the specific application needs; balancing speed, security, and system limitations,

While RSA is widely used, ECC is emerging as a strong alternative due to its reliance on the elliptic curve discrete logarithm problem (ECDLP), which is harder to solve than RSA's integer factorization problem (IFP). ECC offers higher security with smaller key sizes and requires less memory, making it more efficient and suitable for devices with limited resources.[9]

Why RSA is Widely used?[13]

RSA is called a one-way trapdoor function, what it actually means is that there is no way of undoing the encryption unless we know the trapdoor. The trapdoor is the number n which is the product of two large prime numbers (n=p mod q). While it is easy to multiply two large prime numbers to get n, it is extremely difficult to reverse the process and factor n back into p and q. This difficulty is bases on the Fundamental Theorem of Arithmetic, which states that every number has a unique prime factorization. Because the finding the prime factor of a large composite number is so hard and time-consuming (potential takes decades), it ensures the security of RSA- encryption. Only if someone knows the factors of n can decrypt the message.

Distributed Ledger over Traditional Databases

Relational databases, which have been dominant since the 1970s, offer high efficiency, scalability, and low latency, allowing for rapid, concurrent transactions. However, they allow data to be modified, making them non-immutable by default. In contrast, blockchain technology is inherently immutable, designed to prevent alteration of stored data, making it ideal for applications where data integrity is critical.

Despite its strengths, blockchain comes with trade-offs: it has high latency, limited transaction throughput, and expensive data storage. For instance, Bitcoin restricts block sizes to 1 MB, making it impractical for storing large files like medical images. Instead of storing raw data directly, a common solution is to store hashes of the data (such as DICOM images), which ensures integrity while minimizing storage demands.

This limitation has spurred interest in hybrid storage models, where blockchains store only hashes or references, and the actual data resides in off-chain storage systems—potentially cloud-based or decentralized networks. Technologies like IPFS, Storj, Filecoin, and BigChainDB offer such decentralized storage, combining blockchain's immutability with scalable file storage. These solutions incentivize data hosting through tokens and could be more suitable for image-intensive applications like PACS in healthcare, though further research is needed to ensure security, compliance, and long-term feasibility.

Limitations

While blockchain provides us with large number of features, it has its own limitations[11].

Complexity-With an ever-increasing number of blockchain implementation utilizing several underlying technologies the ability of different system to work together will suffer

Privacy and Confidentiality-In public blockchain the data are vulnerable to exposure. If the encryption is broken all the data stored in the blockchain could be exposed.

Speed and Scalability-Blockchain are considered to be slower than the traditional databases and adding new data is limited by the speed of the underlying consensus mechanism.

Security-Data can be rendered completely unreadable if the public key are lost. Additionally, blockchain are at a risk of 51% attack.

Given its current limitations—such as high latency, limited storage capacity, and increased cost—blockchain is unlikely to fully replace traditional database systems like those used for Electronic Medical Records (EMRs), Picture Archiving and Communication Systems (PACS), and Vendor Neutral Archives (VNAs). However, blockchain can serve as a complementary technology, enhancing these systems by improving data security, integrity, and traceability [11].

Applications

Apart from finance Blockchain has a huge application in various field, such fields are:[7]

1. Reducing Data Breach Costs
2. Reducing Data Breach Costs
3. Protecting Intellectual Property
4. Smart Contracts
5. Business Data Privacy and Threats
6. Cybersecurity
7. Supply Chain
8. Protection of Copyright
9. Government

This project underscores the importance of adopting blockchain-based e-voting systems, highlighting their potential to improve security, transparency, and public trust in the electoral process. It specifically centers on a comparative analysis of two encryption algorithms—RSA and ECC—within the framework of a decentralized e-voting platform.

This project presents a Flask-based decentralized e-voting system that has been implemented using both RSA and ECC algorithms for secure vote encryption, signature validation and decryption. The system uses three separate database tables. One table to store voter credentials. Another table to storing admin credentials and the third table for storing the candidate details. The admin table is restricted to only one record. When a user signs up, the system gives them a unique login ID to identify them. Every unique id is generated using token_hex() function present on python secrets library and then it is stored into the database .

On logging in, the users data stores as voters are redirected to the voting ballot where they cast their vote by selecting their preferred candidate. The admin can perform functions like adding candidate, viewing results, viewing blockchain data as well as decrypting the data.
Votes are encrypted using admin public key and only the admin can decrypt the vote using their private key.

In this project the blockchain used are only for demonstration purpose and not the real blockchain. The blocks are created using python dictionaries and are stored in Json file. The block data include block number, time at which it was created(timestamp),voter id, encrypted vote, voter's signature hash of previous block and hash of the present block. The hashes of the block are generated using python sha-256() function present in hashlib library.
A comparative analysis of ECC and RSA is conducted by measuring:

- Key generation time
- Memory usage

The results are visualized using matplotlib which is a python library.

Synopsis

Title- Comparative Analysis of RSA and ECC in a Blockchain-Based E-Voting System

Objective- To develop a secure and transparent electronic voting system that leverages cryptographic techniques such as Elliptic Curve Cryptography (ECC) and Rivest Shamir Adleman (RSA) for encryption, digital signatures, and data integrity. The system ensures vote confidentiality, authenticity, and verifiability, while allowing performance comparison between ECC and RSA.

Modules involved are:

1. Voter Module:
    i. Registration
    ii. Vote
2. Admin Module:
    i. Add Candidate
    ii. View result
    iii. View Block Data
3. Blockchain Module:
    i. Create Block
    ii. Verify Blockchain
    iii. Add Block
    iv. Save Block
4. Performance Analysis Module:
    i. Generate ECC and RSA module
    ii. Measure Key Generation Time and Memory Usage
    iii. Visualize comparison using matplotlib

Technologies Used:

- Backend: Python (Flask)
- Frontend: HTML, CSS
- Database: SQLite
- Cryptography: ECC (SECP384R1), RSA (2048-bit)
- Libraries: cryptography, matplotlib, time, base64, flask, secrets, rsa, json, datetime, hashlib, os, sqlite3.

.

Feasibility Study

Technical Feasibility-

Technologies Used:

- Backend: Python (Flask)
- Frontend: HTML, CSS, JavaScript
- Database: SQLite
- Cryptography: ECC (SECP384R1), RSA (2048-bit)
- Libraries: cryptography, matplotlib, time, base64, flask, secrets, rsa, Json, datetime, hashlib, os, sqlite3.

System requirements:

- Minimum hardware requirement (PC or Laptop)
- Python libraries are installed using pip
- No server infrastructure required

Skill Availability:

- Python and Flask are widely used
- Cryptographic algorithm like ECC and RSA are used
-

Behavioural Feasibility-

User Friendly:

- Web interface is simple
- Role based access to voter and admin

Security Trust:

- Votes are encrypted and signed before storing in the blockchain

Adoptability-

- Educational institutions or mock elections would adopt such system

Economic Feasibility-

- Low Cost of development
- No need for expensive infrastructure

Analysis

Existing Systems:

Many countries and governments are beginning to adopt blockchain-based e-voting systems. Nations like Australia, Norway, Estonia, and Switzerland have already conducted official elections through e-voting pilot programs. Since 2013, Estonia has consistently used e-voting in all its elections and censuses. At the same time, countries such as India and Japan are actively working on developing blockchain-powered e-voting systems for future elections. Some countries, including Australia, Germany, Norway, Sierra Leone, and Switzerland, use commercial platforms for their blockchain voting systems. On the other hand, countries like Estonia, Russia, South Korea, and the United States (particularly Washington D.C.) have developed and use their own in-house systems..[12]

Australis- In 2015, New South Wales piloted a blockchain-based e-voting system using the Scytl app. Around 280,000 citizens could vote electronically. Voters registered to get an ID and PIN to cast their vote and received a receipt number to verify it.

Estonia-Estonia introduced electronic e-voting in 2005, allowing secure online voting using electronic voter IDs. By 2013, about 21.2% of voters chose e-voting, which ensured anonymity, voter identity verification, and a partly decentralized process.

Germany-Polyas is used for its parliamentary election. Polyas is the only e-voting software company certified by German Federal Office for Information Security, for its e-voting system.

Russia-In 2014, Russia introduced e-voting and introduced blockchain-based e-voting in 2017 using Waves platform. The system employs Proof of Authority with crash fault tolerance and smart contract for voting rules, registration and verification.

Switzerland- Luxoft is an e-voting system used for national and municipal election incorporating SMS confirmation and mailed security symbols for vote verification.

United States-Some of the U.S. states have adopted blockchain e-voting system for various purposes. Massachusetts used Votez for non-governmental and local election, while Washington D.C.'s blockchain vote-by-mail was cancelled due to public criticism.

Developed system:

Key generation for ECC; the private key is generated using the curve ec.SECP384R1() present in the ec library. Using the private key, the public key is generated. The voters public key is stored in the

database and used to generate signature. The vote is encrypted using admin's public key. The admin's private key is used during decryption. The voter's private key is used to verify the signature.

This project uses Elliptic Curve Diffie-Hellman (ECDH) (ec.ECDH) to derive a shared symmetric key from the voter's public key and admin's private key. The shared key is then processed into an AES key using a key derivation function(HKFD) called the derived keys. The vote is them encrypted using AES-GCM (using python cipher modules) with the derived keys.

The votes are decrypted, when the admin wants to decrypt the vote. The voter's public key and admin's private key is generated using (load_pem_private_key, load_pem_public_key python module). Re-derive the AES keys using ECDH and HKDF. Decrypt the votes using AES-GCM.

Key generation for RSA; Each voter has their own key pair(public key and private key). Keys are generated using rsa.newkeys(2048). The voter's private key and hash function sha-256() is used to generate voter signature(rsa.sign()). The vote is encrypted (rsa.encrypt()) using admin's public key. The encrypted vote is verified using voter's signature and public key. The vote can be decrypted (using rsa.decrypt().decode)only by admins using their private key.

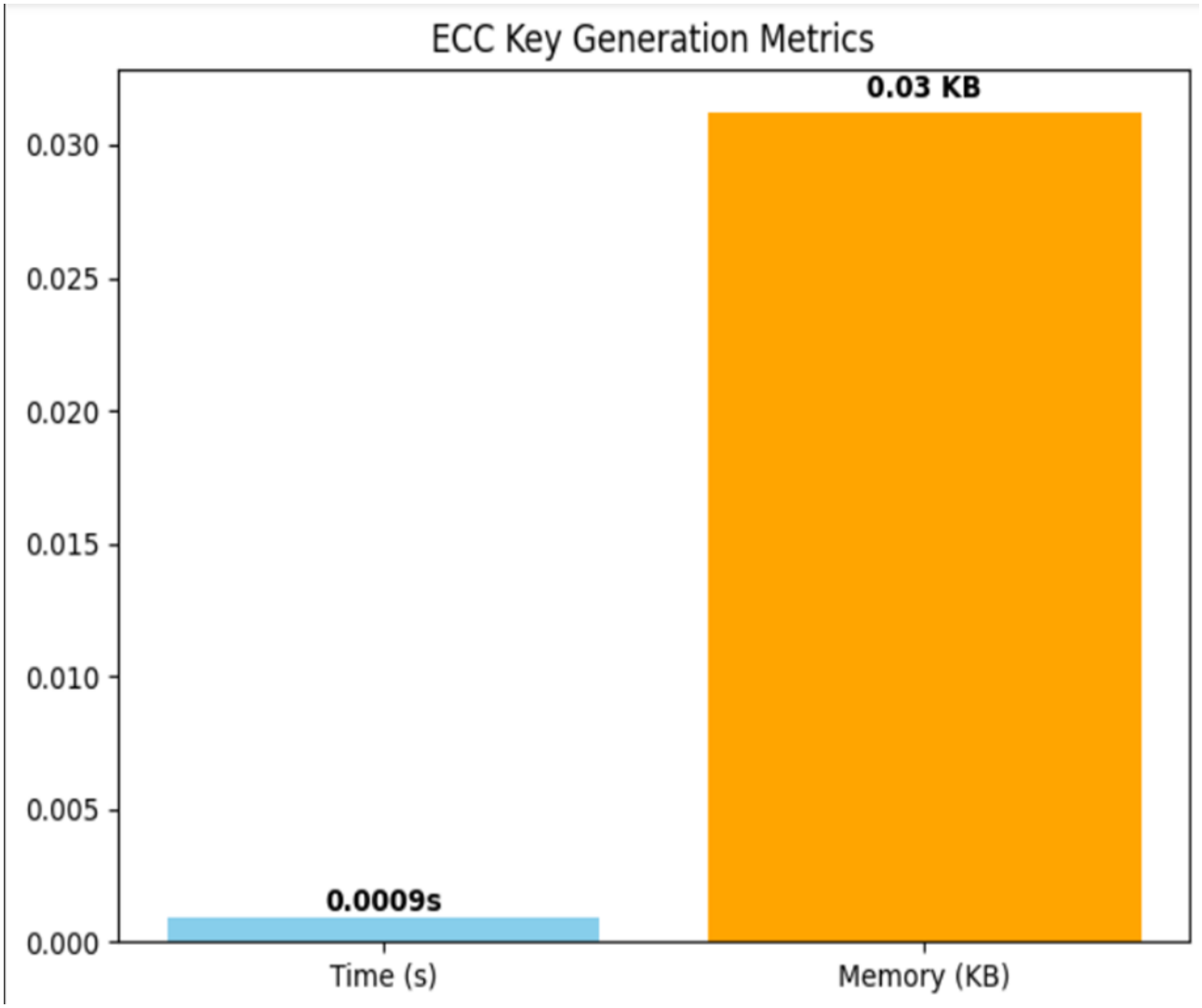


Figure 1.1

The graph titled “ECC Key Generation Metrics” provides a visual overview of ECC (Elliptic Curve Cryptography) performance by showing the time and memory required for key generation.

The y-axis displays values within a small range. Two bars are used: the light blue bar represents the time taken in seconds, while the orange bar indicates memory usage in kilobytes. Each bar is labelled with its precise value.

The graph clearly highlights that ECC key generation is extremely fast—taking only 0.0009 seconds—making it ideal for lightweight and real-time applications

The memory used during the key generation (0.03 KB) is very low making this algorithm suitable for environments with limited resources(e.g., embedded system, IoT)

Thus, to conclude ECC has many advantages such as; efficient key generation time showcasing its speed, consumes very low memory. There criteria make ECC suitable for devices with constrained resources. ECC’s complexity and dependency on secure curves demand careful implemented.

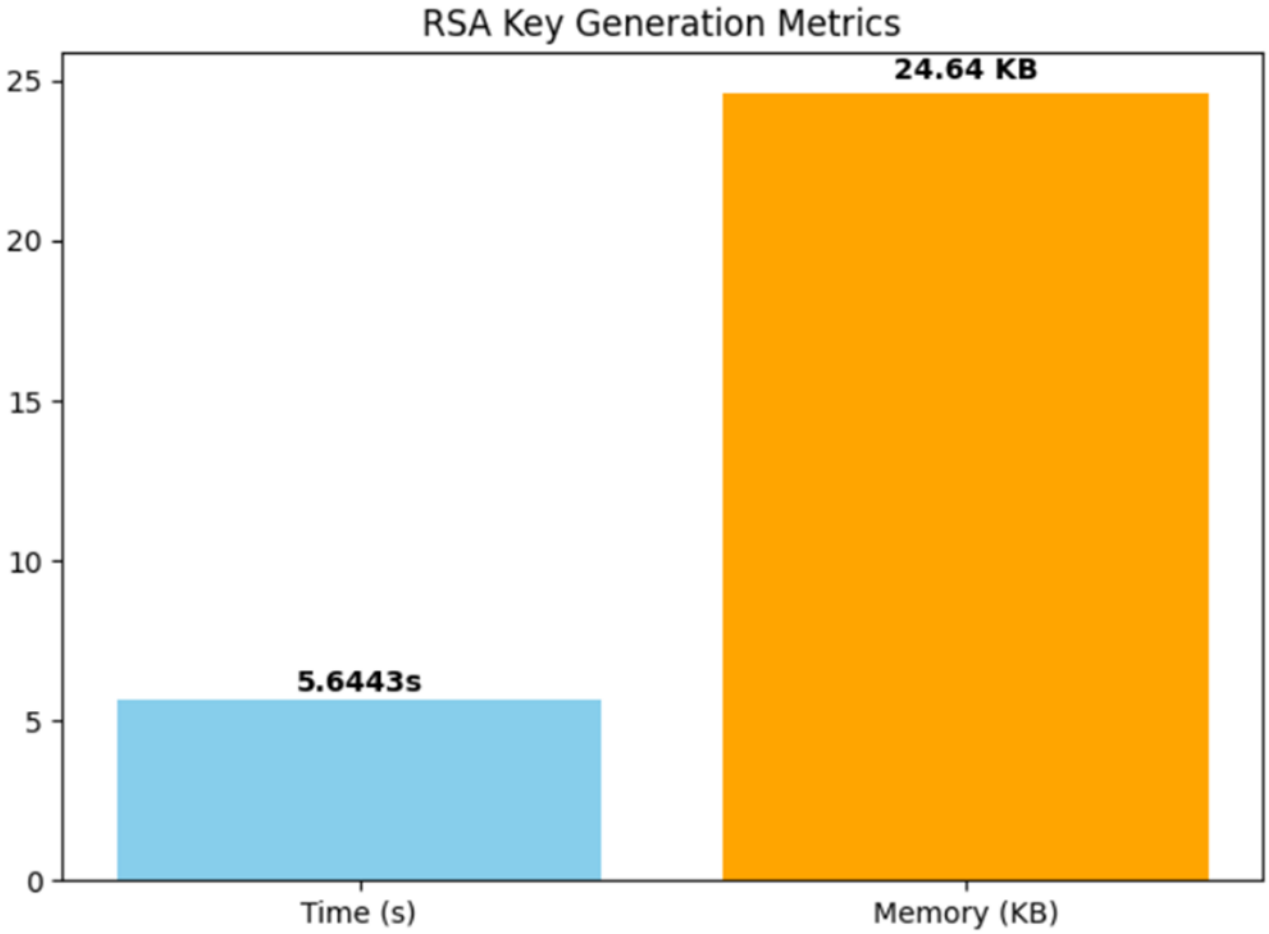


Figure 1.2

The above graph titled" RSA Key Generation Metrics" visually represents the performance of RSA (Rivest Shamir Adleman) in terms of time taken to generate key and memory consumed during the process.

The y-axis represents the values in a small range. The bar that represents time in seconds is shown in light blue color. The bar that represents memory usage in kilobytes is shown in orange color. The bars are labeled with exact values.

The consumed to generate the key is very high. 5.6 seconds is significantly high for real-time and resource constrained devices. Suggesting RSA not suitable for quick key generation.

The memory usage (24.64 KB) is moderate but much higher than the lightweight cryptographic algorithms like ECC.

Thus, the graph highlights RSA's computational heaviness, both in terms of time and memory, for key generation. While RSA offer robust security, its performance overhead makes it less suitable for low-power or real-time applications.

Hardware and Software Requirements

Developer requirements:

Hardware requirement:

- Processor: Intel core i5/i7 or equivalent (quad core recommended)
- RAM: 8 GB or more
- Storage: At least 256 GB
- Network: Stable internet for downloading dependencies.

Software Requirement:

- Operating system: Windows 10/11, macOS, or any Linux distribution.
- Python: Version 3.12 or higher
- Code Editor/IDE: VS Code
- Browser: Chrome for testing
- Libraries
  - Flask (web framework)
  - SQLite (lightweight database)
  - cryptography for ECC
  - rsa module for RSA
  - matplotlib for graph
  - Base64, secrets, Json, datetime, hashlib, os.

User requirement:

Hardware requirement:

- Processor: Dual core or better
- RAM: 4 GB or more
- Storage: At least 100 MB
- Display: Basic Monitor

Software requirement:

- Operating system: Windows, Linux, macOS
- Bowser: Any Browser(Chrome, Firefox, Edge)
- No installation is required if the app is hosted, users can access it using a browser

Design

The ECC(Elliptic Curve Cryptography) Encryption Algorithm[12]

Key Generation:

1. Start
2. Select a random number dA between the range [1,n-1]. dA is the private key of the sender.
3. Calculate public key of the sender dB= dA*G, here G is the generator point on the curve where the operation starts.
4. Stop

Signature Generation:

1. Start
2. Calculate cryptographic hash function e=HASH(message).
3. Sender selects a random integer k from [1,n-1).
4. Compute (r,s).
5. r= x1(mod n), (x1, y1)=k*G.
6. s=k-1(e+dA*r)
7. (r,s) is the signature.

Encryption:

1. Start
2. Let m be the message is any point M on the elliptic curve.
3. Select a random number k from [1,n-1].
4. The cipher text generated will be pair of points(b1,b2), b1=k*G and b2=M+(k*G).
5. Stop

Decryption:

1. Start
2. Receiver compute product of b1 and b2.
3. Subtract this product from the point b2. M=b2-(b1*b2), M is the plain text.
4. Stop

Flowchart

ECC Key generation

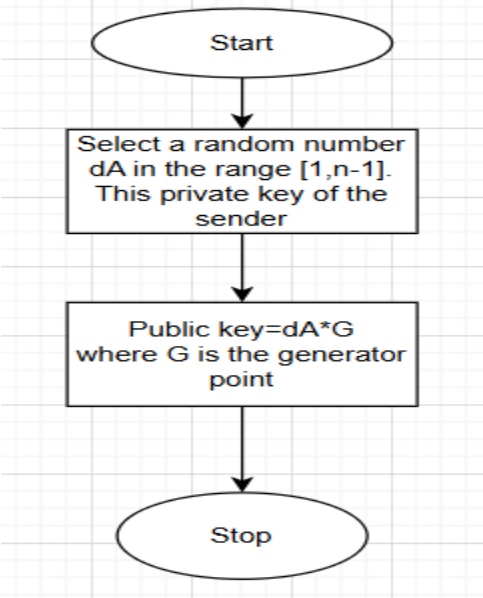


Figure 1.3

ECC Signature Generation

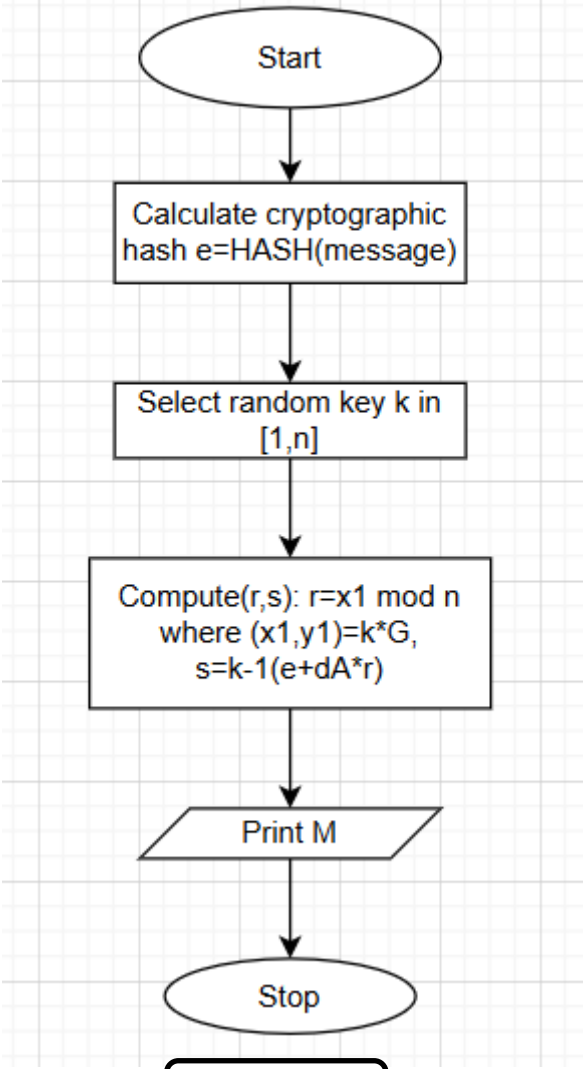


Figure 1.4

ECC Encryption

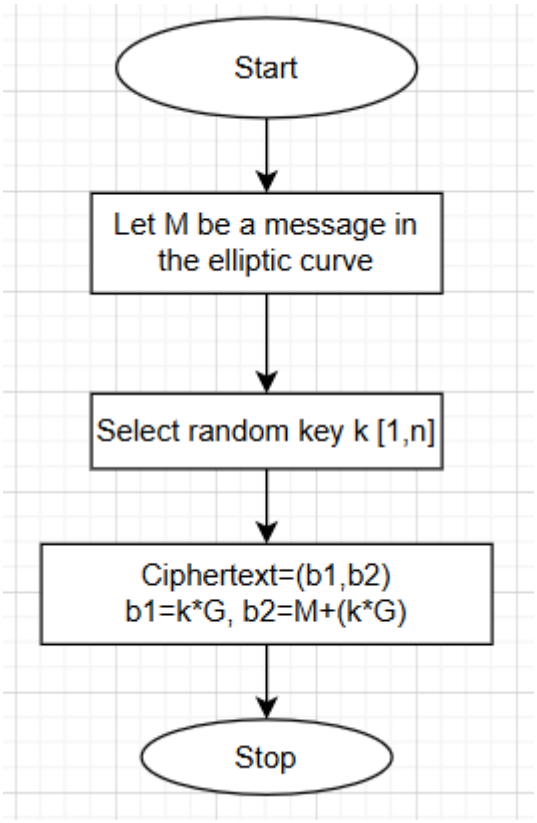


Figure 1.5

ECC Decryption

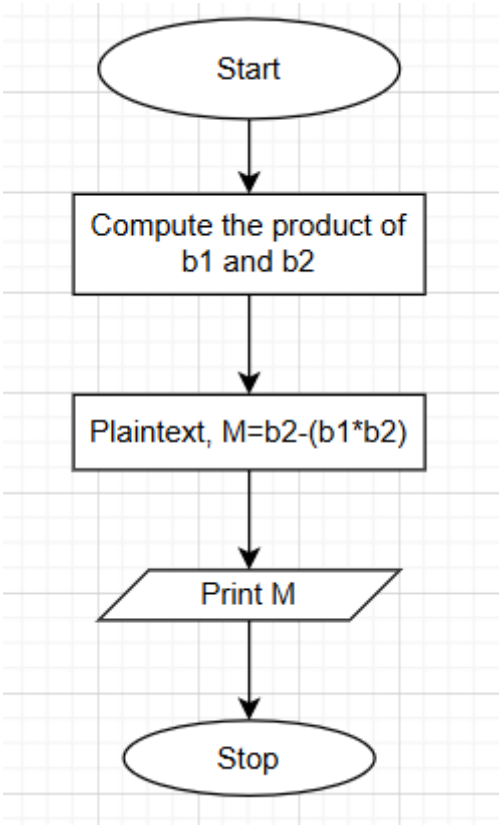


Figure 1.6

The RSA (Rivest Shamir Adleman) Encryption Algorithm[4]

Key Generation

1. Start
2. Choose any two prime number p,q. From these numbers we can calculate the modulus, n=pq
3. Select a third number, e that is relatively prime to the product (p-1)(q-1), the number e is the public exponent.
4. Calculate an integer d from the quotient $\frac{ed-1}{(p-1)(q-1)}$. d is the private exponent.
5. Stop

Encryption

1. Start
2. Read the plaintext, the two selected prime numbers(p,q), e(public exponent) and d(private exponent).
3. Select a number e such that it is relatively prime to the product (p-1)(q-1).
4. Calculate an integer d from the quotient $\frac{ed-1}{(p-1)(q-1)}$.
5. To encrypt a plaintext M, use the equation, C=$M^e mod\ n$, where C is the cipher text.
6. Print the cipher text C.
7. Stop

Decryption

1. Start.
2. Read the cipher text C, the mod of the two prime numbers(n=pq), and d which is an integer derived from the quotient $\frac{ed-1}{(p-1)(q-1)}$.
3. To decrypt the ciphertext use the equation M=$C^d mod\ n$, where M is the plaintext.
4. Print M.
5. Stop.

Flowchart

RSA Key generation

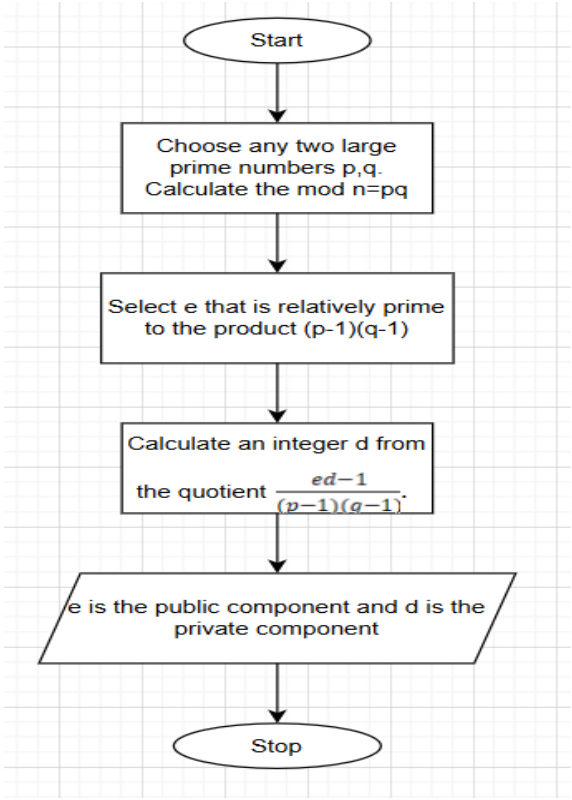


Figure 1.7

RSA Encryption

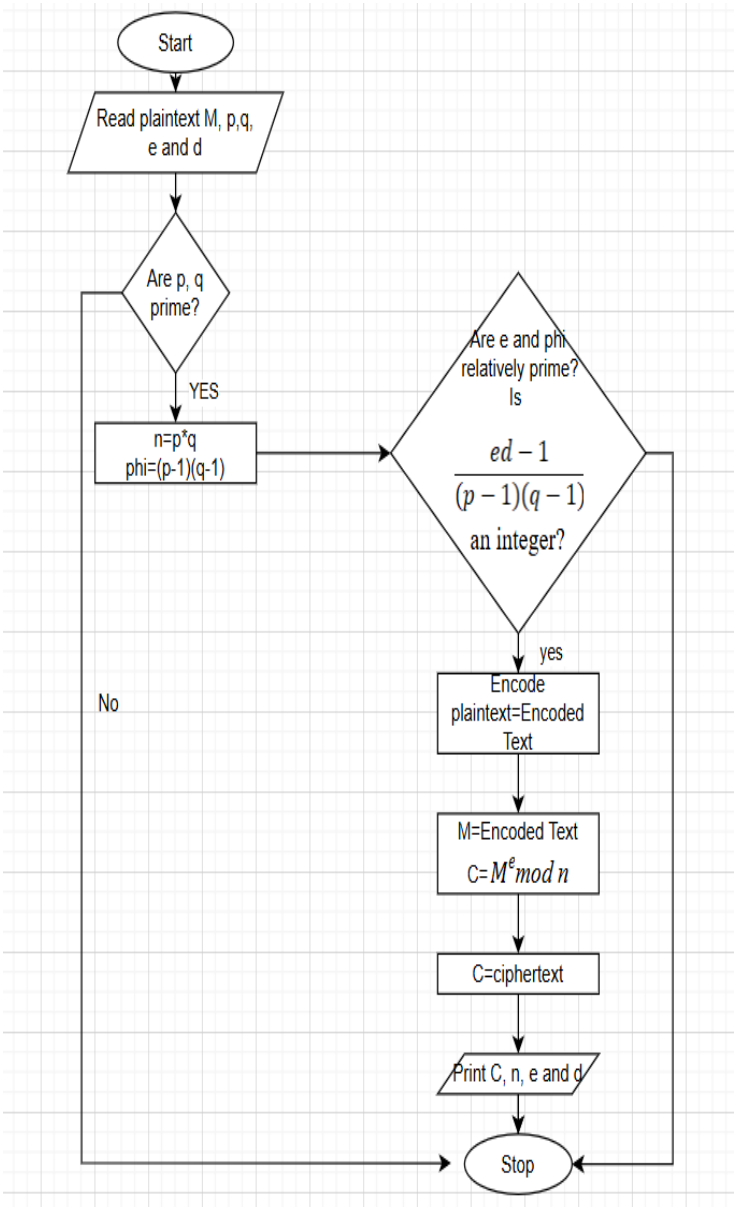


Figure 1.8

RSA Decryption

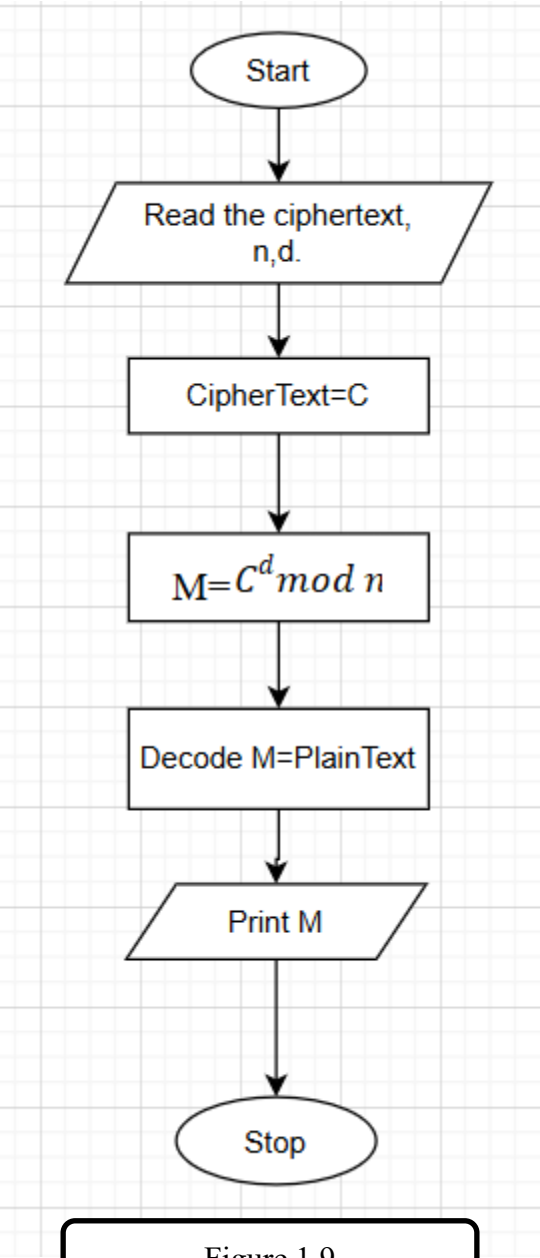
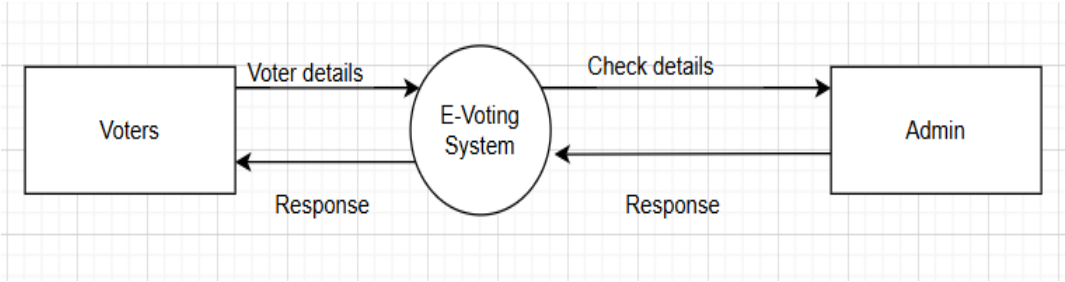
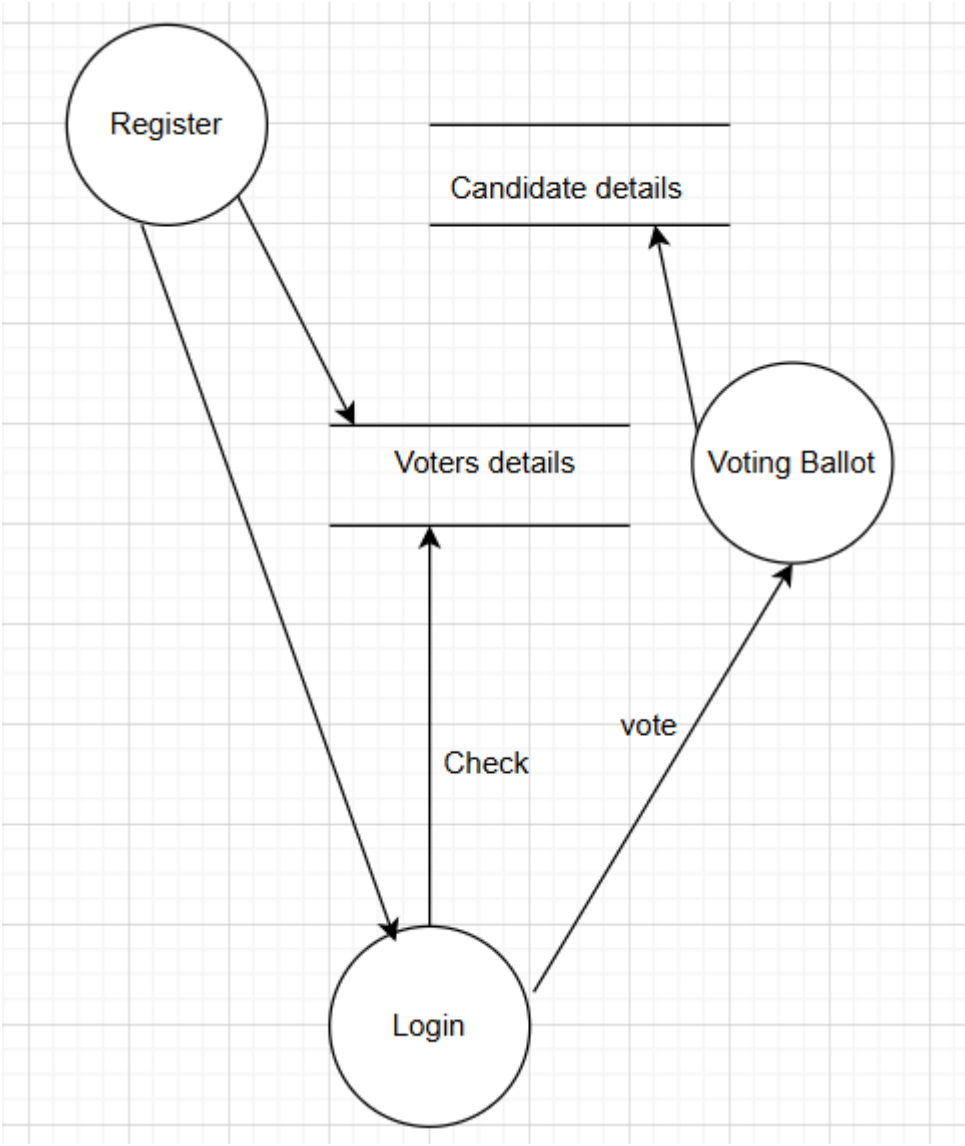


Figure 1.9

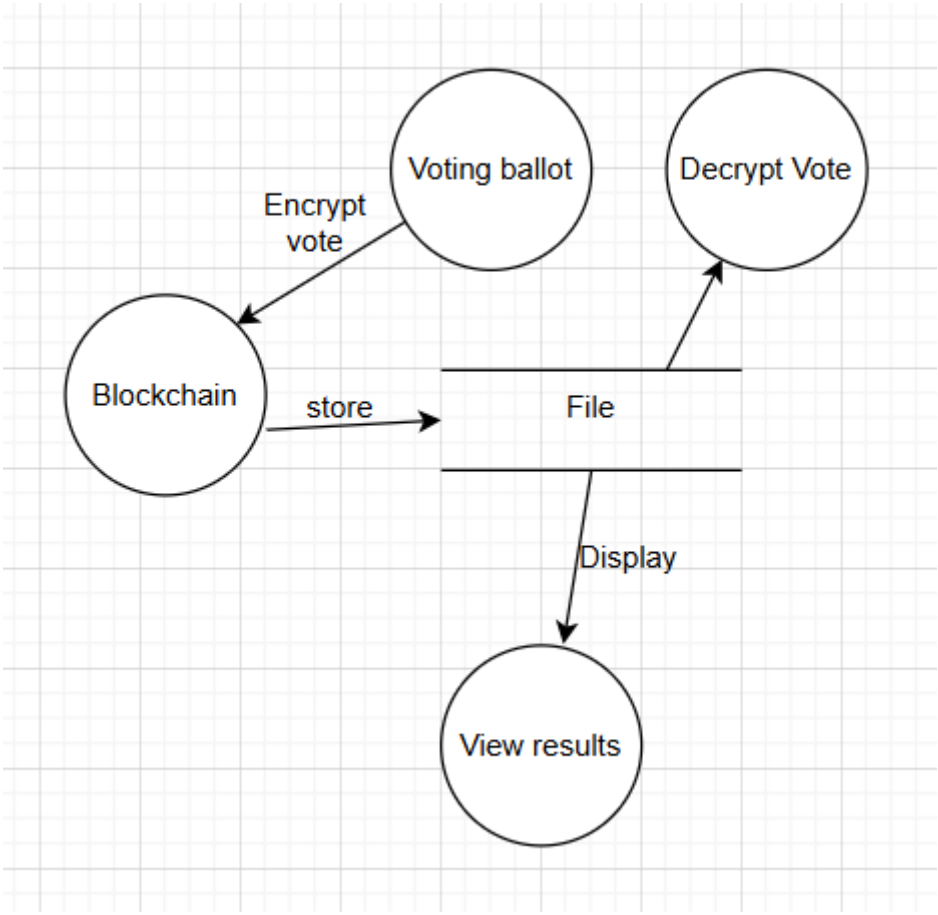
DFD



Level 0 DFD



Level 1 DFD


Voting ballot
Decrypt Vote
Encrypt
vote
Blockchain
store
File
Display
View results

Level 2 DFD

Future Study

1. Comparative study with other algorithms
    a. Extending the comparative study on symmetric encryption algorithm and hash-based algorithm.
    b. Measuring the performance of the encryption algorithm not only using key time generation and memory but all encryption and decryption time for different security levels.
2. Biometric Authentication
    a. This project can be escalated by adding fingerprint, facial recognition, or iris scan for voter login.
3. Integrate with Blockchain
    a. Incorporate full blockchain functionalities to store encrypted votes immutably. Use smart contract to validate the votes and automatic results computation.
4. Real-Time Monitoring Admin Dashboard
    a. Create an admin dashboard with real-time statistics, visualization, and alerts for anomalies.
5. User Interface Enhancement
    a. To enhance the front end with responsive design and screen reader compatibility.

Screenshots of Developed System

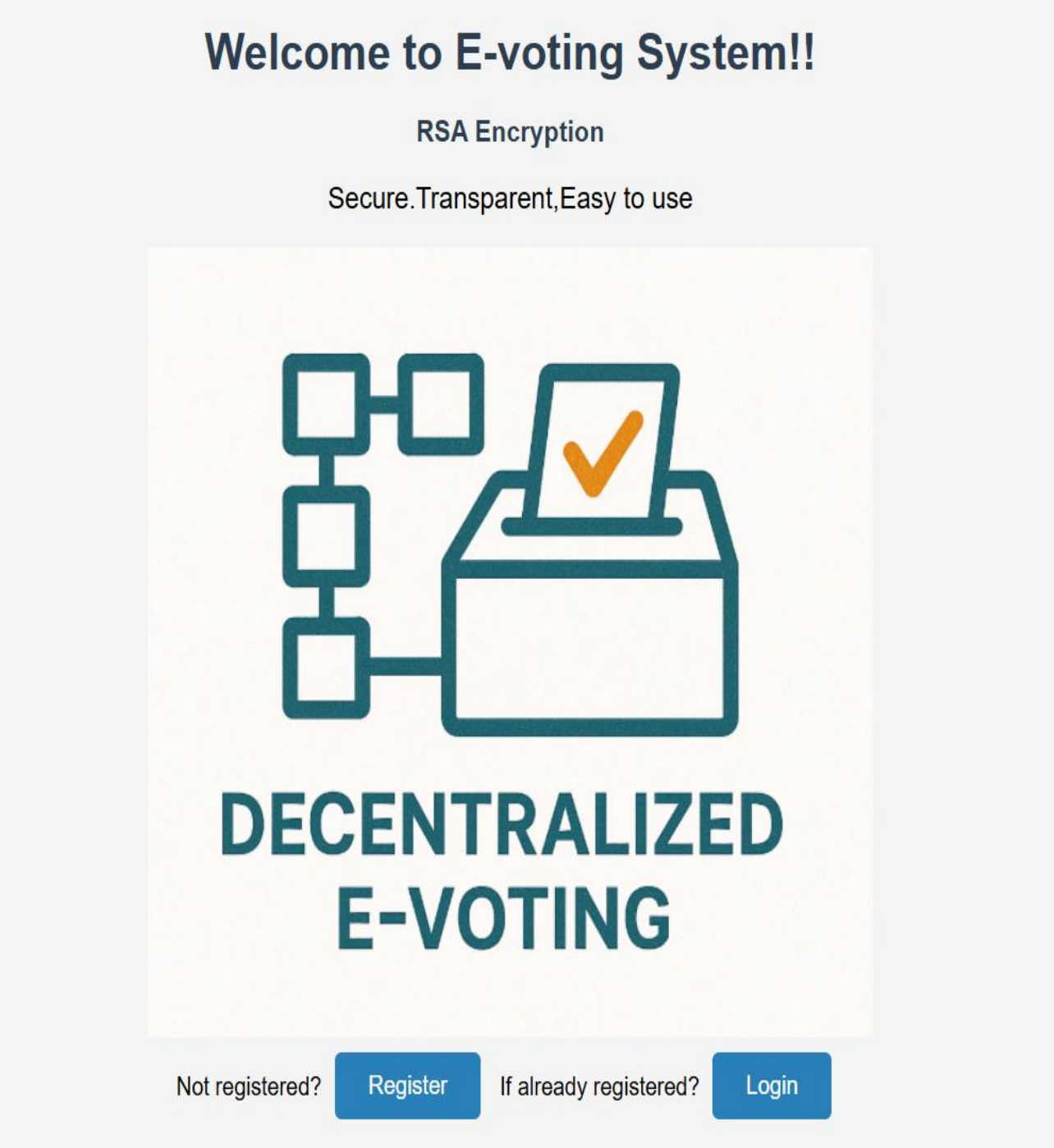


Figure 1.10 Home Page(RSA)

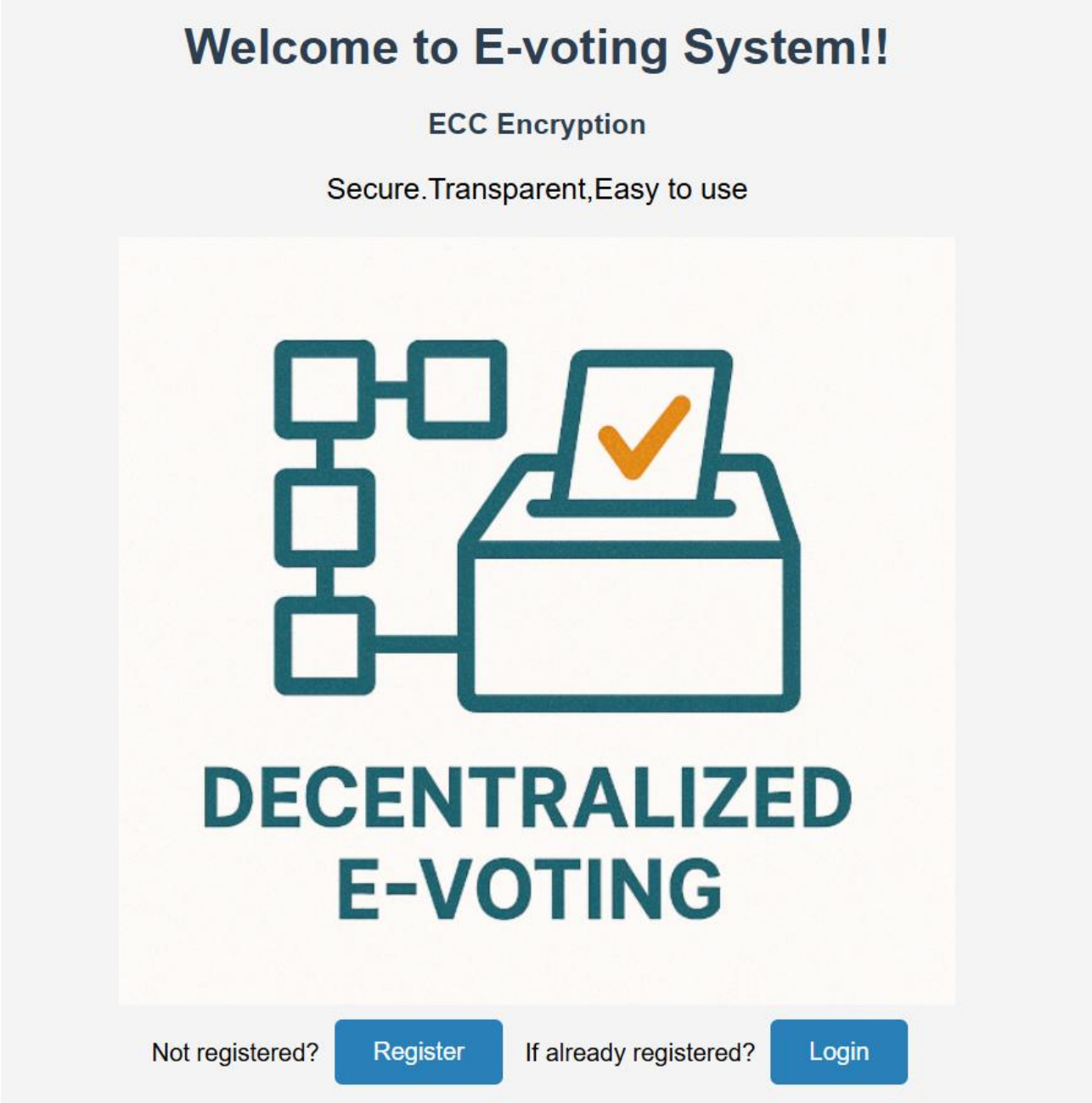


Figure 1.11 Home Page (ECC)

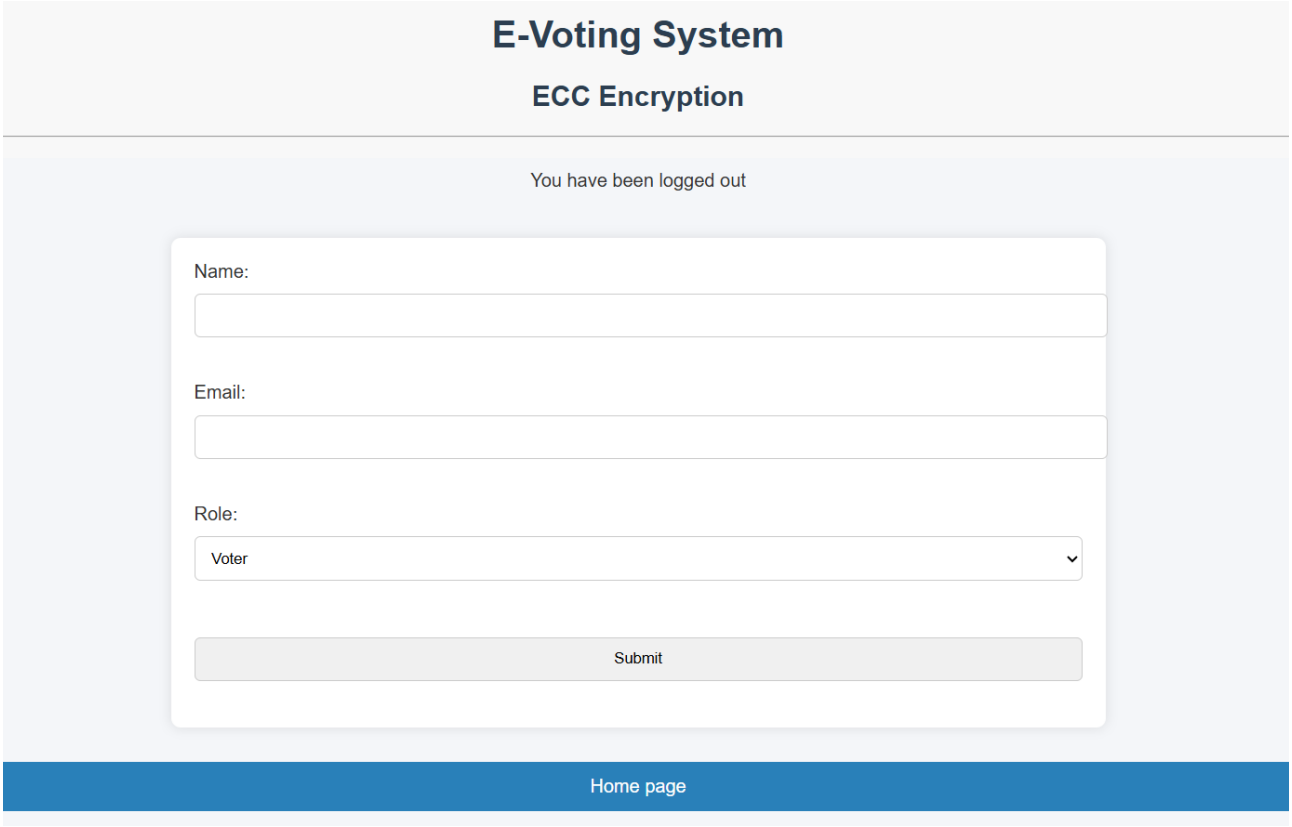


Figure 1.12 Registration Page (ECC)

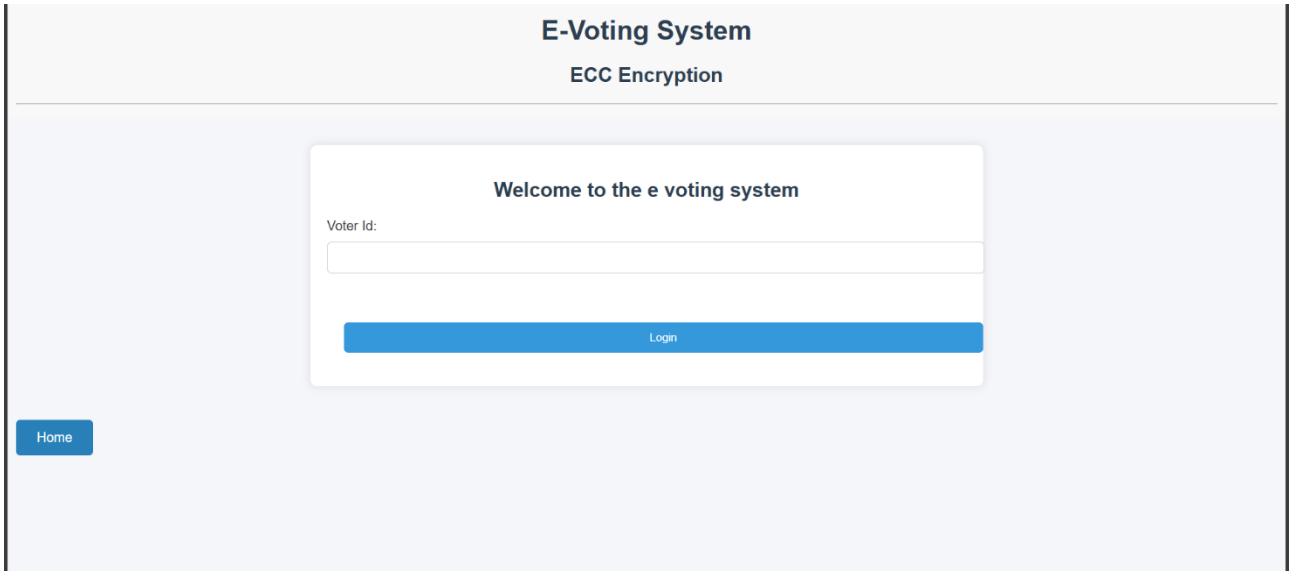


Figure 1.13 Login Page(ECC)

Conclusion

The implementation of blockchain based e-voting system has shown how ECC and RSA algorithms can be used to develop a secure, transparent and tamper-resistant approach to modern electronic voting. By leveraging public key cryptography, the system ensures that the votes remain secure, authentic and verifiable throughout the process. The comparative analysis done for both ECC and RSA has revealed that though RSA is widely accepted and used, ECC is far faster and more efficient. ECC with low time to generate key and moderate memory usage proved to be resource-efficient while maintaining strong security. This project has well demonstrated the integration of cryptographic methods with blockchain can efficiently mitigate vote-tampering. This solution can make the future digital processes both scalable and trustworthy. Thus highlighted the potential of cryptography and blockchain in transforming democratic process.

Code

Project Folder-Blockchain based e-voting system using ECC encryption

- app.py
- blockchain.py
- ecc_utils.py
- templates
  - add_candidate.html
  - admin_dashboard.html
  - base.html
  - benchmark.html
  - block_data.html
  - decrypt.html
  - form.html
  - login.html
  - reg_success.html
  - result.html
  - vote_success.html
  - voting_ballot.html
  - Welcome.html
- Static
  - Css
  - images

app.py

```
from flask import Flask, request, render_template,flash,session,redirect,url_for
import sqlite3
import secrets
from Blockchain import blockchain
import base64
from datetime import datetime
from sqlite3 import IntegrityError
from ecc_utils import *
from cryptography.hazmat.primitives.serialization import load_pem_private_key,load_pem_public_key
import json
import time

bchain=blockchain()
app = Flask(__name__)
app.secret_key="a91fd9e4b60b40bfa7763d39d124b87b" # some fatures in flask needs the secret_key
DATABASE = 'Vote.db'

def get_db_connection():
    conn = sqlite3.connect(DATABASE)
    conn.row_factory = sqlite3.Row  # This makes the rows behave like dictionaries
    return conn

def create_table():
    conn = get_db_connection()
    cursor = conn.cursor()
    cursor.execute('''CREATE TABLE IF NOT EXISTS admin(
                    id INTEGER PRIMARY KEY AUTOINCREMENT,
                    name TEXT NOT NULL,
                    email TEXT NOT NULL,
                    role TEXT NOT NULL CHECK (role IN ('admin', 'voter')),
                    voter_id TEXT UNIQUE NOT NULL,
                    public_key TEXT NOT NULL,
                    private_key TEXT NOT NULL)''')
    cursor.execute('''CREATE TRIGGER IF NOT EXISTS admin_limit
                    BEFORE INSERT ON admin
                    WHEN(SELECT COUNT(*) FROM admin)>=1
                    BEGIN
                       SELECT RAISE(ABORT,'ONLY ONE ADMIN ALLOWED');
                    END;''')
    cursor.execute('''CREATE TABLE IF NOT EXISTS voters (
                    id INTEGER PRIMARY KEY AUTOINCREMENT,
                    name TEXT NOT NULL,
                    email TEXT NOT NULL,
                    role TEXT NOT NULL CHECK (role IN ('admin', 'voter')),
                    voter_id TEXT UNIQUE NOT NULL)''')
    cursor.execute('''CREATE TABLE IF NOT EXISTS Candidates(
                    id INTEGER PRIMARY KEY AUTOINCREMENT,
                    name TEXT NOT NULL,
                    party TEXT NOT NULL,
                    description TEXT,
                    image TEXT,
                     vote_count INTEGER DEFAULT 0)''')
```

```
conn.commit()
  conn.close()

create_table()

@app.route("/")
def Welcome():
    return render_template('Welcome.html')

def gen_time():
    start=time.time()

    private_key=ec.generate_private_key(ec.SECP384R1())
    public_key=private_key.public_key()

    #serialise to text-based format to store in the database
    pri_pem=private_key.private_bytes(
        encoding=serialization.Encoding.PEM,
    format=serialization.PrivateFormat.PKCS8,
    encryption_algorithm=serialization.NoEncryption()
    ).decode()

    pub_pem=public_key.public_bytes(
        encoding=serialization.Encoding.PEM,
        format=serialization.PublicFormat.SubjectPublicKeyInfo
    ).decode()

    end=time.time()

    keygen_time=end-start
    return keygen_time
import tracemalloc
def gen_men_usage():
    tracemalloc.start()
    for _  in range(10):
        private_key=ec.generate_private_key(ec.SECP384R1())
        public_key=private_key.public_key()

        #serialise to text-based format to store in the database
        pri_pem=private_key.private_bytes(
            encoding=serialization.Encoding.PEM,
        format=serialization.PrivateFormat.PKCS8,
        encryption_algorithm=serialization.NoEncryption()
        ).decode()

        pub_pem=public_key.public_bytes(
            encoding=serialization.Encoding.PEM,
            format=serialization.PublicFormat.SubjectPublicKeyInfo
        ).decode()
        current,peak=tracemalloc.get_traced_memory()
    tracemalloc.stop()
    return peak/1024

@app.route('/form', methods=['GET','POST'])
def form():
```

```
if request.method == 'POST':
        name = request.form['name']
        email = request.form['email']
        role=request.form.get('role','voter') # by default role will be voter




            private_key=ec.generate_private_key(ec.SECP384R1())
         public_key=private_key.public_key()

         #serialise to text-based format to store in the database
         pri_pem=private_key.private_bytes(
          encoding=serialization.Encoding.PEM,
         format=serialization.PrivateFormat.PKCS8,
         encryption_algorithm=serialization.NoEncryption()
         ).decode()

         pub_pem=public_key.public_bytes(
            encoding=serialization.Encoding.PEM,
            format=serialization.PublicFormat.SubjectPublicKeyInfo
         ).decode()


         if not name or not email or not role:
            flash("PLease enter all the fields",'error')
            return redirect(url_for('form'))

         conn = get_db_connection()
         cursor = conn.cursor()
         #check if admin or not
         if role.lower()=='admin':
            voter_id=secrets.token_hex(4)
            try:
               cursor.execute('INSERT INTO admin(name,email,role,voter_id,public_key,private_key)
VALUES(?,?,?,?,?,?)',(name,email,role,voter_id,pub_pem,pri_pem))
               conn.commit()
               flash('Data inserted successfully!')
               return render_template('reg_success.html', voter_id=voter_id)
            except IntegrityError as e:
               if 'ONLY ONE ADMIN ALLOWED' in str(e):
                  flash('Only one admin is allowed')
               else:
                  flash(f'Error: {str(e)}')
            finally:
               conn.close()
         else:
            #check if the user already exist
            cursor.execute('SELECT * FROM voters WHERE name=? AND email=?',(name,email))
            if cursor.fetchone():
               conn.close()
               return redirect(url_for('Welcome'))
            else:
               voter_id=secrets.token_hex(4)
               cursor.execute(
```

```
 'INSERT INTO voters (name, email,role,voter_id,has_voted,public_key,private_key) VALUES
(?,?,?,?,?,?,?)', (name, email,role,voter_id,0,pub_pem,pri_pem))
            conn.commit()  # Save the changes
            conn.close()
            flash('Data inserted successfully!')
            return render_template('reg_success.html', voter_id=voter_id)

    return render_template('form.html')




@app.route('/login',methods=['GET','POST'])
def login():
    if request.method=='POST':
        voter_id=request.form.get('voter_id')

        conn=get_db_connection()
        cursor=conn.cursor()
        #check admin
        cursor.execute('SELECT * FROM admin WHERE voter_id=?',(voter_id,))
        admin=cursor.fetchone()
        if admin:
            session['user_id']=admin[0]
            session['username']=admin[1]
            session['voter_id']=admin[4]
            session['admin']=True
            flash('Welcome admin')
            conn.close()
            return redirect(url_for('admin_dashboard'))

        #check voter
        cursor.execute('SELECT * FROM voters WHERE voter_id=?',(voter_id,))
        voter=cursor.fetchone()
        if voter:
            session['user_id'] = voter[0]
            session['username'] = voter[1]
            session['role'] = voter[3]
            session['voter_id']=voter_id
            session['has_voted']=voter[5]
            conn.close()
            flash ('Login successful','success')

            if session['role'].lower()=='voter' and voter['has_voted'] == 1:
                flash('You have already voted.')
                return redirect(url_for('Welcome'))
            else:
                return redirect(url_for('voting_ballot'))
        return 'Please generate correct voter id'

    return render_template('login.html')

@app.route('/add_candidate', methods=['GET', 'POST'])
```

```
def add_candidate():

    if request.method == 'POST':
        name = request.form['name']
        party = request.form['party']
        description = request.form['description']
        image = request.form['image']  # Assume filename or URL

        conn = get_db_connection()
        cursor = conn.cursor()









cursor.execute(
            'INSERT INTO Candidates (name, party, description, image) VALUES (?, ?, ?, ?)',
            (name, party, description, image)
        )
        conn.commit()



conn.close()

        flash('Candidate added successfully!', 'success')
        return redirect(url_for('add_candidate'))
    return render_template('add_candidate.html')

@app.route('/admin_dashboard')
def admin_dashboard():
    # Render admin dashboard
    return render_template('admin_dashboard.html')


@app.route('/voting_ballot', methods=['GET', 'POST'])
def voting_ballot():

    user_id = session.get('user_id')
    if request.method == 'POST':
        selected_c_id = request.form.get('candidate')
        if not selected_c_id:
            flash('Please select a candidate', 'error')
            return render_template('voting_ballot.html')

        conn = get_db_connection()
        conn.row_factory = sqlite3.Row
        cursor = conn.cursor()

        cursor.execute('SELECT private_key, public_key, has_voted FROM voters WHERE id=?', (user_id,))
        voter = cursor.fetchone()
        if voter is None:
            flash('User not registered', 'error')
```

```
 conn.close()
            return render_template('voting_ballot.html')

        if voter['has_voted']:
            flash('You have already voted', 'error')
            conn.close()
            return render_template('voting_ballot.html')

        user_private_key = serialization.load_pem_private_key(
            voter['private_key'].encode(), password=None
        )
        user_public_key = serialization.load_pem_public_key(
            voter['public_key'].encode()
        )
        # Get admin public key (PEM)
        cursor.execute('SELECT public_key FROM admin WHERE id=1')
        admin_row = cursor.fetchone()
        if admin_row is None:




flash('Admin public key not found', 'error')
            conn.close()
            return render_template('voting_ballot.html')

        admin_public_key = serialization.load_pem_public_key(
            admin_row['public_key'].encode()
        )
        conn.close()

        aes_key = derive_symmetric_key(user_private_key, admin_public_key)




# Prepare vote data to encrypt
        vote_data = f"{user_id}:{selected_c_id}".encode()

        iv = os.urandom(12)
        encryptor = Cipher(algorithms.AES(aes_key), modes.GCM(iv)).encryptor()
        ciphertext = encryptor.update(vote_data) + encryptor.finalize()
        tag = encryptor.tag

        # Combine iv + tag + ciphertext
        encrypted_vote = iv + tag + ciphertext
        encrypted_vote_b64 = base64.b64encode(encrypted_vote).decode()

        # Sign the encrypted vote
        signature = sign_data(user_private_key, encrypted_vote)
        signature_b64 = base64.b64encode(signature).decode()

        if not verify_signature(user_public_key, signature, encrypted_vote):
```

```
conn.close()
            flash('Signature verification failed. Vote not accepted.', 'error')
            return render_template('voting_ballot.html')

        # Store encrypted vote and update vote count & voter status
        conn = get_db_connection()
        cursor = conn.cursor()

        cursor.execute('UPDATE candidates SET vote_count=vote_count+1 WHERE id=?', (selected_c_id,))
        cursor.execute('UPDATE voters SET has_voted=1 WHERE id=?', (user_id,))
        conn.commit()
        conn.close()
        # Add block with encrypted vote (no signature here for simplicity)
        bchain.add_block(voter_id=user_id, encrypted_vote=encrypted_vote_b64,signature=signature_b64)

        flash('Your vote has been recorded', 'success')
        return render_template('vote_success.html')

    # GET: show candidates
    conn = get_db_connection()
    cursor = conn.cursor()
    cursor.execute('SELECT * FROM candidates')
    candidates = cursor.fetchall()




conn.close()
    return render_template('voting_ballot.html', candidates=candidates)

@app.route('/vote_success')
def vote_success():
    return render_template('vote_success.html')

@app.route('/result')
def result():
    conn=get_db_connection()
    cursor=conn.cursor()
    cursor.execute('SELECT * FROM Candidates')
    result_data=cursor.fetchall()
    cursor.close()
    return render_template('result.html',r_data=result_data)




@app.route('/block_data')
def block_data():
    chain_data=bchain.generate_chain()
    return render_template('block_data.html',chain=chain_data)

import json
@app.route('/decrypt/<int:index>', methods=['GET'])
def decrypt(index):
```

```
 if not session.get('admin'):
        flash('Admin access required', 'error')
        return redirect(url_for('login'))

    try:
        with open('new_blockchain.json', 'r') as f:
            blockchain = json.load(f)
    except Exception as e:
        flash(f'Blockchain could not be loaded: {e}', 'error')
        return redirect(url_for('block_data'))

    if index < 0 or index >= len(blockchain):
        flash('Invalid block index', 'error')
        return redirect(url_for('block_data'))

    block = blockchain[index]
    encrypted_vote_b64 = block.get('encrypted_vote')
    voter_id = block.get('voter_id')

    if not encrypted_vote_b64 or not voter_id:
        flash('Encrypted vote or voter ID not found in block', 'error')
        return redirect(url_for('block_data'))

    # Get keys from DB
    conn = get_db_connection()
    conn.row_factory = sqlite3.Row
    cursor = conn.cursor()




cursor.execute('SELECT private_key FROM admin WHERE id=1')
    admin_row = cursor.fetchone()

    cursor.execute('SELECT public_key FROM voters WHERE id=?', (voter_id,))
    voter_row = cursor.fetchone()
    conn.close()

    if not admin_row or not voter_row:
        flash('Admin or voter key not found', 'error')
        return redirect(url_for('block_data'))

    try:
        # Load admin's private key (SECP384R1)
        admin_private_key = load_pem_private_key(admin_row['private_key'].encode(), password=None)
        # Load voter's public key
        voter_public_key = load_pem_public_key(voter_row['public_key'].encode())

        # Derive AES key
         aes_key = derive_symmetric_key(admin_private_key, voter_public_key)
```

```
# Decode and split the encrypted vote
        encrypted_vote_bytes = base64.b64decode(encrypted_vote_b64)
        iv = encrypted_vote_bytes[:12]
        tag = encrypted_vote_bytes[12:28]
        ciphertext = encrypted_vote_bytes[28:]

        # Decrypt using AES-GCM
        decryptor = Cipher(algorithms.AES(aes_key), modes.GCM(iv, tag)).decryptor()
        decrypted_vote_bytes = decryptor.update(ciphertext) + decryptor.finalize()
        decrypted_vote = decrypted_vote_bytes.decode('utf-8')

        return render_template('decrypt.html', vote=decrypted_vote, block_index=index)

    except Exception as e:
        flash(f'Failed to decrypt vote: {repr(e)}', 'error')
        return redirect(url_for('block_data'))
@app.route('/logout')
def logout():
    session.clear()
    flash('You have been logged out','info')
    return render_template("Welcome.html")

@app.template_filter('datetimeformat')
def datetimeformat(value):
    return datetime.fromtimestamp(value).strftime('%Y-%m-%d %H:%M:%S')

@app.route('/benchmark')
def show_benchmark():
    ecc_keygen_time = gen_time()  # Only ECC keygen time with serialization
    ecc_men_usage=gen_men_usage()
    # You can add RSA benchmark or others here too




 return render_template(
        'benchmark.html',
        ecc_keygen=ecc_keygen_time,
        ecc_men_usage=ecc_men_usage
    )


if __name__ == '__main__':
    app.run(debug=True)
```

Blockchain.py

```
import time
import hashlib
import Json
import os

class Block:
    def __init__(self,index,timestamp,voter_id,encrypted_vote,signature,previous_hash):
        self.index=index
        self.timestamp=timestamp
        self.voter_id=voter_id
        self.encrypted_vote=encrypted_vote
        self.signature=signature
        self.previous_hash=previous_hash
        self.hash=self.calculate_hash()

    def calculate_hash(self):
        block_str=json.dumps ({
            'index':self.index,
            'timestamp': self.timestamp,
            'voter_id' : self.voter_id,
            'encrypted_vote': self.encrypted_vote,
            'signature' : self.signature,
            'previous_hash' : self.previous_hash
            }, sort_keys=True).encode()
        return hashlib.sha256(block_str).hexdigest()

    def to_dict(self):
        return {
            'index': self.index,
            'timestamp': self.timestamp,
            'voter_id': self.voter_id,
            'encrypted_vote': self.encrypted_vote,
            'signature': self.signature,
            'previous_hash': self.previous_hash,
            'hash': self.hash
        }

    def from_dict(block_dict):
        block = Block(
            block_dict['index'],
            block_dict['timestamp'],
            block_dict['voter_id'],
            block_dict['encrypted_vote'],
            block_dict['signature'],
            block_dict['previous_hash']
        )
        # hash is calculated in __init__, but to keep it consistent, overwrite it
        block.hash = block_dict['hash']
        return block

class blockchain:
    def __init__(self, filename='new_blockchain.json'):
        self.filename = filename
        self.chain = []
```

```
self.load_chain()

def genesis_block(self):
        gen_block = Block(0, time.time(), "0", "GENESIS", "GENESIS", "0")
        self.chain.append(gen_block)
        self.save_chain()

    def load_chain(self):
        if os.path.exists(self.filename):
            if os.stat(self.filename).st_size == 0:
                print("Empty blockchain.json — generating genesis block.")
                self.genesis_block()
            else:
                with open(self.filename, 'r') as f:
                    data = json.load(f)
                    self.chain = [Block.from_dict(block) for block in data]
        else:
            self.genesis_block()
            self.save_chain()

    def save_chain(self):
        with open(self.filename, 'w') as f:
            json.dump([block.to_dict() for block in self.chain], f, indent=4)


    def add_block(self, voter_id, encrypted_vote, signature):
        last_block = self.chain[-1]
        new_block = Block(
            index=len(self.chain),
            timestamp=time.time(),
            voter_id=voter_id,
            encrypted_vote=encrypted_vote,
            signature=signature,
            previous_hash=last_block.hash
        )
        self.chain.append(new_block)
        self.save_chain()

    def generate_chain(self):
        return [block.to_dict() for block in self.chain]
```

ecc_utils.py

```
from cryptography.hazmat.primitives.asymmetric import ec
from cryptography.hazmat.primitives import hashes, serialization
from cryptography.hazmat.primitives.kdf.hkdf import HKDF
import os

# Load PEM-formatted keys
def load_private_key(pem):
    return serialization.load_pem_private_key(pem.encode(), password=None)

def load_public_key(pem):
    return serialization.load_pem_public_key(pem.encode())

# Derive AES key via ECDH
def derive_symmetric_key(private_key, peer_public_key):
    shared_key = private_key.exchange(ec.ECDH(), peer_public_key)
    derived_key = HKDF(
        algorithm=hashes.SHA256(),
        length=32,
        salt=None,
        info=b'handshake',
    ).derive(shared_key)
    return derived_key

# Encrypt using AES-GCM
def encrypt_vote(symmetric_key, plaintext):
    import os
import base64
from cryptography.hazmat.primitives.ciphers import Cipher, algorithms, modes
from cryptography.hazmat.primitives.kdf.hkdf import HKDF
from cryptography.hazmat.primitives import hashes, serialization
from cryptography.hazmat.primitives.asymmetric import ec, padding
from cryptography.hazmat.primitives.asymmetric.utils import encode_dss_signature

def encrypt_vote(private_key, public_key):
    shared_key = private_key.exchange(ec.ECDH(), public_key)
    return HKDF(
        algorithm=hashes.SHA256(),
        length=32,
        salt=None,
        info=b'handshake data',
    ).derive(shared_key)

def derive_symmetric_key(private_key, peer_public_key):
    shared_key = private_key.exchange(ec.ECDH(), peer_public_key)
    derived_key = HKDF(
        algorithm=hashes.SHA256(),
        length=32,
        salt=None,
        info=b'handshake data',
    ).derive(shared_key)
    return derived_key
# Sign encrypted vote
def sign_data(private_key, message):
    return private_key.sign(message, ec.ECDSA(hashes.SHA256()))
```

```
# Verify signature
def verify_signature(public_key, signature,message):
    try:
        public_key.verify(signature, message, ec.ECDSA(hashes.SHA256()))
        return True
    except Exception:
        return False
```

add_candidate.html

```
<html>
    <head>
        <title>Add Candidate</title>
        <link rel="stylesheet" href="{{ url_for('static', filename='css/style.css') }}">
        <style>
            .button {
            display: inline-block;
            padding: 12px 25px;
            margin: 10px;
            font-size: 16px;
            color: #fff;
            background-color: #2980b9;
            border: none;
            border-radius: 5px;
            text-decoration: none;
        }
        .button:hover {
            background-color: #1f6391;
        }
        </style>
    </head>
    <body>
        {% extends 'base.html' %}

        {% block title %}User Registration{% endblock %}

        {% block content %}

        {% with messages = get_flashed_messages(with_categories=true) %}
            {% if messages %}
                {% for category, message in messages %}
                    <div class="flash {{ category }}">{{ message }}</div>
                {% endfor %}
            {% endif %}
        {% endwith %}
        <form method="POST" action="add_candidate">
            Enter candidate name: <input type="text" name="name"><br>
            Enter party name: <input type="text" name="party"><br>
            Enter party description: <textarea name="description"></textarea><br>
            Enter image of the candidate or the party: <input type="text" name="image"><br>
            <button type="submit">Add Candidate</button>
        </form>
        <a href="{{url_for('admin_dashboard')}}" class="button">Admin_dashboard</a>
        <a href="{{url_for('Welcome')}}" class="button">Home</a>
        {% endblock %}</body></html>
```

admin_dashboard.html

```
<!DOCTYPE html>
<html>
   <head>
      <title>Admin Dashboard</title>
      <link rel="stylesheet" href="{{ url_for('static', filename='css/style.css') }}">
         <style>
            h1{
               align-items: center;
               align-content: center;
               align-self: center;
            }
            a{
               display: inline;
               align-items: center;
               background-color:#2980b9 ;
            }
            .button {
            display: inline-block;
            padding: 12px 25px;
            margin: 10px;
            font-size: 16px;
            color: #fff;
            background-color: #2980b9;
            border: none;
            border-radius: 5px;
            text-decoration: none;
         }
         .button:hover {
            background-color: #1f6391;
         }
      </style>
   </head>
   <body>
      {% extends 'base.html' %}

      {% block title %}User Registration{% endblock %}

      {% block content %}
      <h1>Admin Dashboard</h1>
      <a href="{{url_for('add_candidate')}}" class="button">Add candidate</a>
      <a href="{{url_for('result')}}" class="button">Result</a>
      <a href="{{url_for('block_data')}}" class="button">Block data</a>
      <a href="{{url_for('show_benchmark')}}" class="button">Time</a>
      {% endblock %}
   </body>
</html>
```

base.html

```
<!DOCTYPE html>
<html>
<head>
   <meta charset="UTF-8">
   <title>{% block title %}E-Voting System{% endblock %}</title>
```

```
<link rel="stylesheet" href="{{ url_for('static', filename='style.css') }}">
</head>
<body>

   <!-- Common Header -->
   <header>
      <h1>E-Voting System</h1>
      <h2>ECC Encryption</h2>
      <hr>
      {% if session.get('username') %}
         <div class="logout-section">
            <span>Welcome, {{ session['username'] }}!</span>
            <a href="{{ url_for('logout') }}" class="logout-button">Logout</a>
         </div>
      {% endif %}
   </header>

   <!-- Main Content -->
   <main>
      <div class="content">
         {% with messages = get_flashed_messages(with_categories=true) %}
            {% if messages %}
               {% for category, message in messages %}
                  <div class="flash {{ category }}">{{ message }}</div>
               {% endfor %}
            {% endif %}
         {% endwith %}
         {% block content %}{% endblock %}
      </div>
   </main>
</body>
</html>
```

benchmark.html

```
<!DOCTYPE html>
<html>
<head>
   <title>RSA vs ECC Benchmark</title>
   <style>
      table { border-collapse: collapse; width: 60%; margin: 40px auto; }
      th, td { border: 1px solid #ccc; padding: 12px; text-align: center; }
      th { background-color: #f2f2f2; }
      h2 { text-align: center; margin-top: 30px; }
   </style>
</head>
<body>
   <h2>RSA vs ECC Performance Benchmark </h2>
   <table>
      <thead>
         <tr>
            <th>Operation</th>
            <th>Time ECC (ECDH + AES)</th>
            <th>Memory usage in Kib</th>
         </tr>
```

```
</thead>
<tbody>
        <tr>
          <td>Key Generation</td>
          <td>{{ '%.10f' | format(ecc_keygen) }}</td>
          <td>{{ '%.10f' | format(ecc_men_usage) }}</td>
        </tr>
      </tbody>
  </table>
  <a href="{{url_for('admin_dashboard')}}" class="button">Admin_dashboard</a>
  <a href="{{url_for('Welcome')}}" class="button">Home</a>
</body>
</html>
```

block_data.html

```
<!DOCTYPE html>
<html>
  <head>
    <title>Block Data</title>
    <link rel="stylesheet" href="{{ url_for('static', filename='css/style.css') }}">
    <style>
      table {
        width: 100%;
        border-collapse: collapse;
        font-family: Arial, sans-serif;
      }
      th, td {
        border: 1px solid #ddd;
        text-align: left;
        padding: 8px;
      }
      th {
        background-color: #f2f2f2;
      }
      .button {
      display: inline-block;
      padding: 12px 25px;
      margin: 10px;
      font-size: 16px;
      color: #fff;
      background-color: #2980b9;
      border: none;
      border-radius: 5px;
      text-decoration: none;
    }
    .button:hover {
      background-color: #1f6391;
    }
    </style>

  </head>
  <body>
```

decrypt.html

```
<!DOCTYPE html>
<html>
   <title>Decrypted Vote</title>
      <link rel="stylesheet" href="{{ url_for('static', filename='css/style.css') }}">
      <style>
         body {
         font-family: Arial, sans-serif;
         padding: 20px;
         }
         .container {
            max-width: 600px;
            margin: auto;
            border: 1px solid #ccc;
            border-radius: 10px;
            padding: 30px;
            box-shadow: 0 0 10px rgba(0,0,0,0.1);
         }
         .vote-box {
            background-color: #f9f9f9;
            padding: 15px;
            border-radius: 5px;
            margin-top: 20px;
            font-size: 18px;
            word-wrap: break-word;
         }
         .back-link {
            display: inline-block;
            margin-top: 20px;
            text-decoration: none;
            color: #007BFF;
         }
         .back-link:hover {
            text-decoration: underline;
         }
      </style>
      <body>
         <div class="container">
         <h1>Decoded Vote Data</h1>
         <p>The following vote was successfully decrypted using admin's private key</p>
         <div class="vote-box">
            <strong>Decrypted vote:</strong><br><br>
            {{vote}}
         </div>
         <a href="{{url_for('block_data')}}" class="back-link">Back to Block Data</a>
      </div>
      </body>
</html>
```

form.html

```
<!DOCTYPE html>
<html>
   <head>
      <title>Test</title>
      <link rel="stylesheet" href="{{ url_for('static', filename='css/style.css') }}">
      <style>
         .link-container {
   display: flex;
   justify-content: center;
   gap: 20px; /* space between buttons */
   margin-top: 20px;
}

.button {
   display: grid;
   padding: 12px 25px;
   margin: 0 auto;
   font-size: 16px;
   color: #fff;
   background-color: #2980b9;
   border: none;
   border-radius: 5px;
   text-decoration: none;
   align-content: center;
   align-items: center;
   justify-content: center;
   place-items:center;
}

.button:hover {
   background-color: #0056b3;
}

      </style>
   </head>
   <body>
      {% extends 'base.html' %}

      {% block title %}User Registration{% endblock %}

      {% block content %}
      <form method="post" action="/form">
         <label for="name">Name:</label>
         <input type="text" id="name" name="name"><br><br>

         <label for="email">Email:</label>
         <input type="email" id="email" name="email"><br><br>

         <label for="role">Role:</label>
         <select name="role" id="role" required>
            <option value="voter">Voter</option>
            <option value="admin">Admin</option>
         </select><br><br>
```

```
<input type="submit" value="Submit">

        </form>
        <a href="{{url_for('Welcome')}}" class="button">Home page</a>
        {% endblock %}
    </body>
</html>
```

login.html

```
<!DOCTYPE html>
<html>
    <head>
        <title>Login</title>
        <link rel="stylesheet" href="{{ url_for('static', filename='css/style.css') }}">
        <style>
            .button {
            display: inline-block;
            padding: 12px 25px;
            margin: 10px;
            font-size: 16px;
            color: #fff;
            background-color: #2980b9;
            border: none;
            border-radius: 5px;
            text-decoration: none;
        }
        .button:hover {
            background-color: #1f6391;
        }
        </style>
    </head>
    <body>
        {% extends 'base.html' %}

        {% block title %}User Registration{% endblock %}

        {% block content %}
        {% with messages = get_flashed_messages(with_categories=true) %}
            {% if messages %}
                <ul class="flashes">
                    {% for category, message in messages %}
                        <li class="{{ category }}">{{ message }}</li>
                    {% endfor %}
                </ul>
            {% endif %}
        {% endwith %}

        <form method="POST" action="login">
            <h2>Welcome to the e voting system</h2>
            {% if error %}
                <p style="color: red;">{{error}}</p>
            {% endif %}
            <label>Voter Id:</label>
            <input type="text" name="voter_id" required><br><br>
```

```
<button type="submit">Login</button>
        </form>
        <a href="{{url_for('Welcome')}}" class="button">Home</a>
        {% endblock %}
    </body>
</html>
```

retg_success.html

```
<!DOCTYPE html>
<html>
<head>
    <title>Registration Successful</title>
    <link rel="stylesheet" href="{{ url_for('static', filename='css/style.css') }}">
    <style>
        .button {
            display: inline-block;
            padding: 12px 25px;
            margin: 10px;
            font-size: 16px;
            color: #fff;
            background-color: #2980b9;
            border: none;
            border-radius: 5px;
            text-decoration: none;
        }
        .button:hover {
            background-color: #1f6391;
        }
    </style>
</head>
<body>
     {% extends 'base.html' %}

    {% block title %}User Registration{% endblock %}

    {% block content %}
    <h2>Registration Successful!</h2>
    <p>Your Unique Voter ID is: <strong>{{ voter_id }}</strong></p>
    <p>Please keep this ID safe. It will be used for login and voting.</p>
    <a href="{{url_for('login')}}" class="button">Go to login</a>
    <a href="{{url_for('Welcome')}}" class="button">Home</a>
    {% endblock %}
</body>
</html
```

result.html

```
<!DOCTYPE html>
<html>
    <head>
        <title>Results</title>
```

```
<link rel="stylesheet" href="{{ url_for('static', filename='css/style.css') }}">
      <style>
         .button {
         display: inline-block;
         padding: 12px 25px;
         margin: 10px;
         font-size: 16px;
         color: #fff;
         background-color: #2980b9;
         border: none;
         border-radius: 5px;
         text-decoration: none;
      }
      .button:hover {
         background-color: #1f6391;
      }
      </style>
   </head>
   <body>
      {% extends 'base.html' %}

      {% block title %}User Registration{% endblock %}

      {% block content %}
      <h2>Voting Results</h2>
      <table>
         <thead>
            <tr>
               <td>Candidate name</td>
               <td>Party</td>
               <td>Vote count</td>
            </tr>
         </thead>
         <tbody>
            {% for c in r_data %}
               <tr>
                  <td>{{c['name']}}</td>
                  <td>{{c['party']}}</td>
                  <td>{{c['vote_count']}}</td>
               </tr>
            {% endfor %}
         </tbody>
      </table>
      <a href="{{url_for('Welcome')}}" class="button">Home</a>
      {% endblock %}
   </body>
</html>
```

vote_success.html

```
<!DOCTYPE html>
<html>
   <head>
      <title>Vote success</title>
      <link rel="stylesheet" href="{{ url_for('static', filename='css/style.css') }}"><style>
```

```
.button {
        display: inline-block;
        padding: 12px 25px;
        margin: 10px;
        font-size: 16px;
        color: #fff;
        background-color: #2980b9;
        border: none;
        border-radius: 5px;
        text-decoration: none;
    }
    .button:hover {
        background-color: #1f6391;
    }
    </style>
  </head>
  <body>
    {% extends 'base.html' %}

    {% block title %}User Registration{% endblock %}

    {% block content %}
      <h1>Thanks for voting</h1>
      <h2>Your vote has been recorded successfully!!</h2>
      <a href="{{url_for('Welcome')}}">Back to Home</a>
      <a href="{{url_for('result')}}" class="button">Result</a>
    {% endblock %}
  </body>
</html>
```

voting_ballot.html

```
<!DOCTYPE html>>
<html>
  <head>
    <title>Voting Ballot</title>
    <link rel="stylesheet" href="{{ url_for('static', filename='css/style.css') }}">
  </head>
  <body>
     {% extends 'base.html' %}

    {% block title %}User Registration{% endblock %}

    {% block content %}
    <h3> Vote for your candidate </h3>
    {% with messages = get_flashed_messages(with_categories=true) %}
      {% if messages %}
        {% for category, message in messages %}
          <div class="flash {{ category }}">{{ message }}</div>
        {% endfor %}
      {% endif %}
    {% endwith %}
```

```
<form method="POST">
        <table>
          <th>Select</th>
          <th>Name</th>
          <th>Party</th>
          <th>Description</th>
          <th>Image</th>
          {% for c in candidates %}
            <tr>
              <td>
                <input type="radio" name="candidate" value="{{c['id']}}">
              </td>
              <td>
                {{ c['name'] }}
              </td>
              <td>
                {{ c['party'] }}
              </td>
              <td>
                {{ c['description'] }}
              </td>
              <td>
                {% if c['image'] %}
                  <img src="{{ url_for('static', filename='images/' ~ c['image']) }}" width="100">
                {% else %}
                  No image
                {% endif %}
              </td>
            </tr>
          {% endfor %}
        </table>
        <button type="submit">Submit Vote</button>
      </form>
      {% endblock %}
  </body>
</html>
```

Welcome.html

```
<!DOCTYPE html>
<html>
  <head>
    <meta charset="UTF-8">
    <title>Welcome to E-voting System</title>
    <style>
      body {
      font-family: Arial, sans-serif;
      margin: 0;
      background-color: #f4f4f4;
      text-align: center;
    }
    .header-image {
```

```
width: 100%;
            max-height: 400px;
            object-fit: cover;
        }
        .container {
            padding: 40px;
        }
        h1,h3 {
            color: #2c3e50;
        }
        p {
            font-size: 1.2em;
            margin: 20px 0;
        }
        .button {
            display: inline-block;
            padding: 12px 25px;
            margin: 10px;
            font-size: 16px;
            color: #fff;
            background-color: #2980b9;
            border: none;
            border-radius: 5px;
            text-decoration: none;
        }
        .button:hover {
            background-color: #1f6391;
        }
        </style>
    </head>
    <body>
        <h1>Welcome to E-voting System!!</h1>
        <h3>ECC Encryption</h3>
        <p>Secure.Transparent,Easy to use</p>

        <img src="{{url_for('static',filename='images/Evoting.png')}}"
        alt="Welcome banner"
        style="width: 100%; max-height: 500px; object-fit: contain; display: block; margin: 0 auto;">
        Not registered?<a href="{{ url_for('form') }}" class="button">Register</a>
        If already registered?<a href="{{ url_for('login') }}" class="button">Login</a>
    </body>
</html>
```

Project Folder-Blockchain based e-voting system using ECC encryption

- app.py
- blockchain.py
- templates
  - add_candidate.html
  - admin_dashboard.html
  - base.html
  - benchmark.html
  - block_data.html
  - decrypt_vote.html
  - form.html
  - login.html
  - reg_success.html
  - result.html
  - vote_success.html
  - voting_ballot.html
  - Welcome.html
- Static
  - Css
  - images

app.py

```
from flask import Flask, request, render_template,flash,session,redirect,url_for
import sqlite3
import secrets
from Blockchain import blockchain
import rsa
from sqlite3 import IntegrityError
import json
from datetime import datetime
import time

bchain=blockchain()
app = Flask(__name__)
app.secret_key="a91fd9e4b60b40bfa7763d39d124b87b" # some fatures in flask needs the secret_key

DATABASE = 'Vote.db'

def get_db_connection():
    conn = sqlite3.connect(DATABASE)
    conn.row_factory = sqlite3.Row  # This makes the rows behave like dictionaries
    return conn

def create_table():
    conn = get_db_connection()
    cursor = conn.cursor()
    cursor.execute('''CREATE TABLE IF NOT EXISTS admin(
                    id INTEGER PRIMARY KEY AUTOINCREMENT,
                    name TEXT NOT NULL,
                    email TEXT NOT NULL,
                    role TEXT NOT NULL CHECK (role IN ('admin', 'voter')),
                    voter_id TEXT UNIQUE NOT NULL,
                    public_key TEXT NOT NULL,
                    private_key TEXT NOT NULL)''')
    cursor.execute('''CREATE TRIGGER IF NOT EXISTS admin_limit
                    BEFORE INSERT ON admin
                    WHEN(SELECT COUNT(*) FROM admin)>=1
                    BEGIN
                       SELECT RAISE(ABORT,'ONLY ONE ADMIN ALLOWED');
                    END;''')
    cursor.execute('''CREATE TABLE IF NOT EXISTS voters (
                    id INTEGER PRIMARY KEY AUTOINCREMENT,
                    name TEXT NOT NULL,
                    email TEXT NOT NULL,
                    role TEXT NOT NULL CHECK (role IN ('admin', 'voter')),
                    voter_id TEXT UNIQUE NOT NULL,
                    has_voted INTEGER DEFAULT 0)''')
    cursor.execute('''CREATE TABLE IF NOT EXISTS Candidates(
                    id INTEGER PRIMARY KEY AUTOINCREMENT,
                    name TEXT NOT NULL,
                    party TEXT NOT NULL,
                    description TEXT,
                    image TEXT,
                    vote_count INTEGER DEFAULT 0)''')
    conn.commit()
    conn.close()
```

```
create_table()

#verify the signed encrypted vote
def verify_vote(encrypted_vote,signature,v_public_key):
    try:
        rsa.verify(encrypted_vote,signature,v_public_key)
        return True
    except rsa.VerificationError:
        return False

def add_padding(s):
    return s + '=' * (-len(s) % 4)
import tracemalloc
def gen_memory_usage():
    tracemalloc.start()
    public_key,private_key=rsa.newkeys(2048)
    pub_key_pem=public_key.save_pkcs1().decode('utf-8')
    pri_key_pem=private_key.save_pkcs1().decode('utf-8')
    current,peak=tracemalloc.get_traced_memory()
    tracemalloc.stop()
    return peak/1024

def gen_time():

    start=time.time()
    public_key,private_key=rsa.newkeys(2048)
    pub_key_pem=public_key.save_pkcs1().decode('utf-8')
    pri_key_pem=private_key.save_pkcs1().decode('utf-8')

    end=time.time()
    key_gen=end-start
    return key_gen

def measure_t_s():
    results={}
    tracemalloc.start()
    start = time.time()
    public_key,private_key=rsa.newkeys(2048)
    pub_key_pem=public_key.save_pkcs1().decode('utf-8')
    pri_key_pem=private_key.save_pkcs1().decode('utf-8')
    rsa_time = time.time() - start
    rsa_mem, _ = tracemalloc.get_traced_memory()
    tracemalloc.stop()
    results['RSA'] = (rsa_time, rsa_mem / 1024)  # in KB

    return


@app.route("/")
def Welcome():
    return render_template('Welcome.html')

@app.route('/form', methods=['GET','POST'])
def form():
    if request.method == 'POST':
        name = request.form['name']
```

```
email = request.form['email']
        role=request.form.get('role','voter') # by default role will be voter

        #generating keys
        public_key,private_key=rsa.newkeys(2048)
        pub_key_pem=public_key.save_pkcs1().decode('utf-8')
        pri_key_pem=private_key.save_pkcs1().decode('utf-8')

        #checkng all the input fields
        if not name or not email or not role:
            flash("PLease enter all the fields",'error')
            return redirect(url_for('form'))

        #db connection
        conn = get_db_connection()
        cursor = conn.cursor()

        #check if admin or not
        if role.lower()=='admin':
            voter_id=secrets.token_hex(4)
            try:
                cursor.execute('INSERT INTO admin(name,email,role,voter_id,public_key,private_key)
VALUES(?,?,?,?,?,?)',(name,email,role,voter_id,pub_key_pem,pri_key_pem))
                conn.commit()
                flash('Data inserted successfully!')
                return render_template('reg_success.html', voter_id=voter_id)
            except IntegrityError as e:
                if 'ONLY ONE ADMIN ALLOWED' in str(e):
                    flash('Only one admin is allowed')
                else:
                    flash(f'Error: {str(e)}')
            finally:
                conn.close()
        else:
            #check if the user already exist
            cursor.execute('SELECT * FROM voters WHERE name=? AND email=?',(name,email))
            if cursor.fetchone():
                conn.close()
                return redirect(url_for('Welcome'))
            voter_id=secrets.token_hex(4)
            cursor.execute(
                'INSERT INTO voters (name, email,role,voter_id,has_voted,public_key,private_key) VALUES
(?,?,?,?,?,?,?)', (name, email,role,voter_id,0,pub_key_pem,pri_key_pem))
            conn.commit()  # Save the changes
            conn.close()
            flash('Data inserted successfully!')
            return render_template('reg_success.html', voter_id=voter_id)
    return render_template('form.html')


@app.route('/login',methods=['GET','POST'])
def login():
    if request.method=='POST':
        voter_id=request.form.get('voter_id')

        conn=get_db_connection()
```

```
cursor=conn.cursor()

        #check admin
        cursor.execute('SELECT * FROM admin WHERE voter_id=?',(voter_id,))
        admin=cursor.fetchone()
        if admin:
            session['user_id']=admin[0]
            session['username']=admin[1]
            session['voter_id']=admin[4]
            session['admin']=True
            flash('Welcome admin')
            conn.close()
            return redirect(url_for('admin_dashboard'))

        #check voter
        cursor.execute('SELECT * FROM voters WHERE voter_id=?',(voter_id,))
        voter=cursor.fetchone()
        if voter:
            session['user_id'] = voter[0]
            session['username'] = voter[1]
            session['role'] = voter[3]
            session['voter_id']=voter_id
            session['has_voted']=voter[5]
            conn.close()
            flash ('Login successful','success')

            if session['role'].lower()=='voter' and voter['has_voted'] == 1:
                flash('You have already voted.')
                return redirect(url_for('Welcome'))
            else:
                return redirect(url_for('voting_ballot'))
        return 'Please generate correct voter id'

    return render_template('login.html')

@app.route('/add_candidate', methods=['GET', 'POST'])
def add_candidate():

    if request.method == 'POST':
        name = request.form['name']
        party = request.form['party']
        description = request.form['description']
        image = request.form['image']  # Assume filename or URL

        conn = get_db_connection()
        cursor = conn.cursor()
        cursor.execute(
            'INSERT INTO Candidates (name, party, description, image) VALUES (?, ?, ?, ?)',
            (name, party, description, image)
        )
        conn.commit()
        conn.close()

        flash('Candidate added successfully!', 'success')
        return redirect(url_for('add_candidate'))
    return render_template('add_candidate.html')
```

```
@app.route('/admin_dashboard')
def admin_dashboard():
    return render_template('admin_dashboard.html')

@app.route('/voting_ballot',methods=['GET', 'POST'])
def voting_ballot():
    gen_e_time=None
    user_id = session.get('user_id')
    if request.method=='POST':
        selected_c_id=request.form.get('candidate')

        if selected_c_id :
            conn = get_db_connection()
            cursor = conn.cursor()

            cursor.execute('SELECT public_key FROM admin WHERE id=1')
            admin_rec=cursor.fetchone()
            if admin_rec is None:
                flash('Admin does not exist')
                return render_template('form.html')
            #admin public key
            a_pub_key=rsa.PublicKey.load_pkcs1(admin_rec[0].encode('utf-8'))

            cursor.execute('SELECT private_key,public_key FROM voters WHERE id=?',(user_id,))
            voter_rec=cursor.fetchone()
            if voter_rec is None:
                flash('user does not exist')
                return render_template('form.html')
            #voter's public and private key
            pub_key=rsa.PublicKey.load_pkcs1(voter_rec[1].encode())
            pri_key=rsa.PrivateKey.load_pkcs1(voter_rec[0].encode())

            #encrypt the vote using admin public key
            vote_data=selected_c_id.encode()

            e_start=time.time()
            encryptes_vote=rsa.encrypt(vote_data,a_pub_key)
            e_end=time.time()
            gen_e_time=e_end-e_start
            #sign the encrypted vote usning voter's private key
            signature=rsa.sign(vote_data,pri_key,'SHA-256')

            #verify the vote using voter's public key
            if verify_vote(vote_data,signature,pub_key):
                cursor.execute('UPDATE Candidates SET vote_count=vote_count+1 WHERE
id=?',(selected_c_id,))
                cursor.execute('UPDATE voters SET has_voted=1 WHERE id=?',(user_id,))
                conn.commit()
                conn.close()
                #store encrypted vote and signature in blockchain
                bchain.add_block(voter_id=user_id,encrypt_vote=encryptes_vote.hex(),signature=signature.hex())

                flash('Your vote has been recorded','success')
                return render_template('vote_success.html')
        else:
```

```
flash('Please select a candidate','error')

#Show ballot
    conn=get_db_connection()
    cursor=conn.cursor()
    cursor.execute('SELECT * FROM candidates')
    candidates = cursor.fetchall()
    conn.close()

    return render_template('voting_ballot.html', candidates=candidates)

@app.route('/vote_success')
def vote_success():
    return render_template('vote_success.html')

@app.route('/result')
def result():
    conn=get_db_connection()
    cursor=conn.cursor()
    cursor.execute('SELECT * FROM Candidates')
    result_data=cursor.fetchall()
    cursor.close()
    return render_template('result.html',r_data=result_data)

@app.route('/block_data')
def block_data():
    chain_data=bchain.generate_chain()
    return render_template('block_data.html',chain=chain_data)

@app.route('/decrypt_vote/<int:index>',methods=['GET','POST'])
def decrypt_vote(index):
    flash(f"Accessing decrypt_vote route with block_index: {index}")
    if request.method=='GET':
        voter_id=session.get('voter_id')
        if voter_id and session.get('admin'):
            try:
                #open json file and load the block content
                with open('blockchain.json','r') as f:
                    blockchain=json.load(f)
            except Exception as e:
                flash('Blockchain could not be loaded',e)
                return redirect(url_for('block_data'))
            #check if the block exist
            if index < 0 or index >= len(blockchain):
                flash('Block is invalid')
                return redirect(url_for('block_data'))
            #get the encrypted vote from the block
            encrypted_vote=blockchain[index].get('encrypt_vote')
            print("Raw Encrypted Vote:", repr(encrypted_vote))
            print("EV",encrypted_vote)
            if not encrypted_vote:
                flash('Encrypted vote not found')
                return redirect(url_for('block_data'))
            #convert back to bytes
            encrypted_vote=blockchain[index].get('encrypt_vote')
            encry_s=encrypted_vote.strip()
```

```
encrypted_vote_b=bytes.fromhex(encry_s)
        #Load admin's private key
        conn=get_db_connection()



         cursor=conn.cursor()
        cursor.execute('SELECT private_key FROM admin WHERE id=1')
        rec=cursor.fetchone()
        conn.close()

        if not rec:
            flash('Admin does not exist')
            return redirect(url_for('block_data'))

        pri_key_pem = rec[0]
        private_key = rsa.PrivateKey.load_pkcs1(pri_key_pem.encode('utf-8'))
        #decrypt the vote
        try:
            decrypt_vote=rsa.decrypt(encrypted_vote_b,private_key).decode('utf-8')
        except Exception as e:
            flash('Decryption failed: ' + str(e))
            return redirect(url_for('block_data'))
        return render_template('decrypt_vote.html',vote=decrypt_vote)
    flash('access denied only admins')
    return redirect(url_for('block_data'))

@app.route('/logout')
def logout():
    session.clear()
    flash('You have been logged out','info')
    return render_template("Welcome.html")

@app.template_filter('datetimeformat')
def datetimeformat(value):
    return datetime.fromtimestamp(value).strftime('%Y-%m-%d %H:%M:%S')

@app.route('/benchmark')
def show_benchmark():
    rsa_keygen_time = gen_time()  # Only ECC keygen time with serialization
    rsa_gen_mem=gen_memory_usage()
    return render_template(
        'benchmark.html',
        rsa_keygen=rsa_keygen_time,
        rsa_gen_mem=rsa_gen_mem
    )


if __name__ == '__main__':
    app.run(debug=True)
```

Blockchain.py

```
import time
import hashlib
import json
import os

class Block:
    def __init__(self,index,timestamp,voter_id,encrypt_vote,signature,previous_hash):
        self.index=index
        self.timestamp=timestamp
        self.voter_id=voter_id
        self.encrypt_vote=encrypt_vote
        self.signature=signature
        self.previous_hash=previous_hash
        self.hash=self.calculate_hash()

    def calculate_hash(self):
        block_str=json.dumps ({
            'index':self.index,
            'timestamp': self.timestamp,
            'voter_id' : self.voter_id,
            'encrypt_vote': self.encrypt_vote,
            'signature' : self.signature,
            'previous_hash' : self.previous_hash
            }, sort_keys=True).encode()
        return hashlib.sha256(block_str).hexdigest()

    def to_dict(self):
        return {
            'index': self.index,
            'timestamp': self.timestamp,
            'voter_id': self.voter_id,
            'encrypt_vote': self.encrypt_vote,
            'signature': self.signature,
            'previous_hash': self.previous_hash,
            'hash': self.hash
        }

    def from_dict(block_dict):
        block = Block(
            block_dict['index'],
            block_dict['timestamp'],
            block_dict['voter_id'],
            block_dict['encrypt_vote'],
            block_dict['signature'],
            block_dict['previous_hash']
        )
        # hash is calculated in __init__, but to keep it consistent, overwrite it
        block.hash = block_dict['hash']
        return block

class blockchain:
    def __init__(self, filename='blockchain.json'):
        self.filename = filename
        self.chain = []
```

```
self.load_chain()

    def genesis_block(self):
        gen_block = Block(0, time.time(), "0", "GENESIS", "GENESIS", "0")
        self.chain.append(gen_block)
        self.save_chain()

    def load_chain(self):
        if os.path.exists(self.filename):
            if os.stat(self.filename).st_size == 0:
                print("Empty blockchain.json — generating genesis block.")
                self.genesis_block()
            else:
                with open(self.filename, 'r') as f:
                    data = json.load(f)
                    self.chain = [Block.from_dict(block) for block in data]
        else:
            self.genesis_block()
            self.save_chain()

    def save_chain(self):
        with open(self.filename, 'w') as f:
            json.dump([block.to_dict() for block in self.chain], f, indent=4)


    def add_block(self, voter_id, encrypt_vote, signature):
        last_block = self.chain[-1]
        new_block = Block(
            index=len(self.chain),
            timestamp=time.time(),
            voter_id=voter_id,
            encrypt_vote=encrypt_vote,
            signature=signature,
            previous_hash=last_block.hash
        )
        self.chain.append(new_block)
        self.save_chain()

    def generate_chain(self):
        return [block.to_dict() for block in self.chain]
```

add.candidate.html

```
<html>
  <head>
    <title>Add Candidate</title>
    <link rel="stylesheet" href="{{ url_for('static', filename='css/style.css') }}">
    <style>
      .button {
      display: inline-block;
      padding: 12px 25px;
      margin: 10px;
      font-size: 16px;
      color: #fff;
      background-color: #2980b9;
      border: none;
      border-radius: 5px;
      text-decoration: none;
    }
    .button:hover {
      background-color: #1f6391;
    }
    </style>
  </head>
  <body>
    {% extends 'base.html' %}

    {% block title %}User Registration{% endblock %}

    {% block content %}

    {% with messages = get_flashed_messages(with_categories=true) %}
      {% if messages %}
        {% for category, message in messages %}
          <div class="flash {{ category }}">{{ message }}</div>
        {% endfor %}
      {% endif %}
    {% endwith %}
    <form method="POST" action="add_candidate">
      Enter candidate name: <input type="text" name="name"><br>
      Enter party name: <input type="text" name="party"><br>
      Enter party description: <textarea name="description"></textarea><br>
      Enter image of the candidate or the party: <input type="text" name="image"><br>
      <button type="submit">Add Candidate</button>
    </form>
    <a href="{{url_for('admin_dashboard')}}" class="button">Admin_dashboard</a>
    {% endblock %}
  </body>
</html>
```

admin_dashboard.html

```
<!DOCTYPE html>
<html>
   <head>
      <title>Admin Dashboard</title>
      <link rel="stylesheet" href="{{ url_for('static', filename='css/style.css') }}">
         <style>
            h1{
               align-items: center;
               align-content: center;
               align-self: center;
            }
            a{
               display: inline;
               align-items: center;
               background-color:#2980b9 ;
            }
            .button {
            display: inline-block;
            padding: 12px 25px;
            margin: 10px;
            font-size: 16px;
            color: #fff;
            background-color: #2980b9;
            border: none;
            border-radius: 5px;
            text-decoration: none;
         }
         .button:hover {
            background-color: #1f6391;
         }
      </style>
   </head>
   <body>
      {% extends 'base.html' %}

      {% block title %}User Registration{% endblock %}

      {% block content %}
      <h1>Admin Dashboard</h1>
      <a href="{{url_for('add_candidate')}}" class="button">Add candidate</a>
      <a href="{{url_for('result')}}" class="button">Result</a>
      <a href="{{url_for('block_data')}}" class="button">Block data</a>
      <a href="{{url_for('show_benchmark')}}" class="button">Time</a>
      {% endblock %}
   </body>
</html>
```

base.html

```
<!DOCTYPE html>
<html>
<head>
   <meta charset="UTF-8">
   <title>{% block title %}E-Voting System{% endblock %}</title>
   <link rel="stylesheet" href="{{ url_for('static', filename='style.css') }}">
</head>
<body>

   <!-- Common Header -->
   <header>
      <h1>E-Voting System</h1>
      <h2>RSA Implementation</h2>
      <hr>
      {% if session.get('username') %}
         <div class="logout-section">
            <span>Welcome, {{ session['username'] }}!</span>
            <a href="{{ url_for('logout') }}" class="logout-button">Logout</a>
         </div>
      {% endif %}
   </header>

   <!-- Main Content -->
   <main>
      <div class="content">
         {% with messages = get_flashed_messages(with_categories=true) %}
            {% if messages %}
               {% for category, message in messages %}
                  <div class="flash {{ category }}">{{ message }}</div>
               {% endfor %}
            {% endif %}
         {% endwith %}
         {% block content %}{% endblock %}
      </div>
   </main>
</body>
</html>
```

benchmark.html

```
<!DOCTYPE html>
<html>
<head>
   <title>RSA vs ECC Benchmark</title>
   <style>
      table { border-collapse: collapse; width: 60%; margin: 40px auto; }
      th, td { border: 1px solid #ccc; padding: 12px; text-align: center; }
      th { background-color: #f2f2f2; }
      h2 { text-align: center; margin-top: 30px; }
   </style>
</head>
<body>
   <h2>RSA vs ECC Performance Benchmark (in seconds)</h2>
   <table>
      <thead>
         <tr>
            <th>Operation</th>
            <th>Time </th>
            <th>Memory usage in KiB</th>
         </tr>
      </thead>
      <tbody>
         <tr>
            <td>Key Generation</td>
            <td>{{ '%.10f' | format(rsa_keygen) }}</td>
            <td> {{ '%.2f' | format(rsa_gen_mem) }} </td>
         </tr>

      </tbody>
   </table>
   <a href="{{url_for('admin_dashboard')}}" class="button">Admin_dashboard</a>
   <a href="{{url_for('Welcome')}}" class="button">Home</a>
</body>
</html>
```

block_data.html

```
<!DOCTYPE html>
<html>
  <head>
    <title>Block Data</title>
    <link rel="stylesheet" href="{{ url_for('static', filename='css/style.css') }}">
    <style>
      table {
        width: 100%;
        border-collapse: collapse;
        font-family: Arial, sans-serif;
      }
      th, td {
        border: 1px solid #ddd;
        text-align: left;
        padding: 8px;
      }
      th {
        background-color: #f2f2f2;
      }
      .button {
      display: inline-block;
      padding: 12px 25px;
      margin: 10px;
      font-size: 16px;
      color: #fff;
      background-color: #2980b9;
      border: none;
      border-radius: 5px;
      text-decoration: none;
    }
    .button:hover {
      background-color: #1f6391;
    }
    </style>

  </head>
  <body>
    {% extends 'base.html' %}

    {% block title %}User Registration{% endblock %}

    {% block content %}
    {% with messages = get_flashed_messages(with_categories=true) %}
      {% if messages %}
        {% for category, message in messages %}
          <div class="flash {{ category }}">{{ message }}</div>
        {% endfor %}
      {% endif %}
    {% endwith %}
    <h2>Block data are here</h2>
    <table border="1" cellpadding="10" cellspacing="0">
      <thead>
        <tr>
          <th>Index</th>
```

```
          <th>Timestamp</th>
              <th>Voter ID</th>
              <th>Signature</th>
              <th>Previous Hash</th>
              <th>Hash</th>
              {% if session.get('admin') %}
                 <th>Actions</th>
              {% endif %}
            </tr>
          </thead>
          <tbody>
          {% for block in chain %}
            <tr>
              <td>{{ block.index }}</td>
              <td>{{ block.timestamp | datetimeformat}}</td>
              <td>{{ block.voter_id }}</td>
              <td class="wrap" >{{block.signature}}</td>
              <td class="wrap" >{{ block.previous_hash }}</td>
              <td class="wrap" >{{ block.hash }}</td>
              {% if session.get('admin') %}
              <td>
                 <form method="GET" action="{{ url_for('decrypt_vote', index=loop.index0 ) }}">
                    <button type="submit">Decrypt Vote</button>
                 </form>
              </td>
              {% endif %}
            </tr>
          {% endfor %}
          </tbody>
        </table>
        <a href="{{url_for('admin_dashboard')}}" class="button">Admin_dashboard</a>
        <a href="{{url_for('Welcome')}}" class="button">Home page</a>
        {% endblock %}
    </body>
</html>
```

decrypt_vote.html

```
<!DOCTYPE html>
<html>
  <head>
    <title>Decrypted Vote</title>
    <link rel="stylesheet" href="{{ url_for('static', filename='css/style.css') }}">
    <style>
      body {
      font-family: Arial, sans-serif;
      padding: 20px;
      }
      .container {
        max-width: 600px;
        margin: auto;
        border: 1px solid #ccc;
        border-radius: 10px;
        padding: 30px;
        box-shadow: 0 0 10px rgba(0,0,0,0.1);
      }
      .vote-box {
        background-color: #f9f9f9;
        padding: 15px;
        border-radius: 5px;
        margin-top: 20px;
        font-size: 18px;
        word-wrap: break-word;
      }
      .back-link {
        display: inline-block;
        margin-top: 20px;
        text-decoration: none;
        color: #007BFF;
      }
      .back-link:hover {
        text-decoration: underline;
      }
    </style>
  </head>
  <body>
    <div class="container">
      <h1>Decoded Vote Data</h1>
      <p>The following vote was successfully decrypted using admin's private key</p>
      <div class="vote-box">
        <strong>Decrypted vote:</strong><br><br>
        {{vote}}
      </div>
      <a href="{{url_for('block_data')}}" class="back-link">Back to Block Data</a>
    </div>
  </body>
</html>
```

form.html

```
<!DOCTYPE html>
<html>
   <head>
      <title>Test</title>
      <link rel="stylesheet" href="{{ url_for('static', filename='css/style.css') }}">
      <style>
         .link-container {
   display: flex;
   justify-content: center;
   gap: 20px; /* space between buttons */
   margin-top: 20px;
}

.button {
   display: grid;
   padding: 12px 25px;
   margin: 0 auto;
   font-size: 16px;
   color: #fff;
   background-color: #2980b9;
   border: none;
   border-radius: 5px;
   text-decoration: none;
   align-content: center;
   align-items: center;
   justify-content: center;
   place-items:center;
}

.button:hover {
   background-color: #0056b3;
}

      </style>
   </head>
   <body>
      {% extends 'base.html' %}

      {% block title %}User Registration{% endblock %}

      {% block content %}
      <form method="post" action="/form">
         <label for="name">Name:</label>
         <input type="text" id="name" name="name"><br><br>

         <label for="email">Email:</label>
         <input type="email" id="email" name="email"><br><br>

         <label for="role">Role:</label>
         <select name="role" id="role" required>
            <option value="voter">Voter</option>
            <option value="admin">Admin</option>
         </select><br><br>
```

```
<input type="submit" value="Submit">

        </form>
        <a href="{{url_for('Welcome')}}" class="button">Home page</a>
        {% endblock %}
   </body>
</html>
```

login.html

```
<!DOCTYPE html>
<html>
   <head>
      <title>Login</title>
      <link rel="stylesheet" href="{{ url_for('static', filename='css/style.css') }}">
      <style>
         .button {
         display: inline-block;
         padding: 12px 25px;
         margin: 10px;
         font-size: 16px;
         color: #fff;
         background-color: #2980b9;
         border: none;
         border-radius: 5px;
         text-decoration: none;
      }
      .button:hover {
         background-color: #1f6391;
      }
      </style>
   </head>
   <body>
      {% extends 'base.html' %}

      {% block title %}User Registration{% endblock %}

      {% block content %}
      {% with messages = get_flashed_messages(with_categories=true) %}
         {% if messages %}
            <ul class="flashes">
               {% for category, message in messages %}
                  <li class="{{ category }}">{{ message }}</li>
               {% endfor %}
            </ul>
         {% endif %}
      {% endwith %}

      <form method="POST" action="login">
         <h2>Welcome to the e voting system</h2>
         {% if error %}
            <p style="color: red;">{{error}}</p>
         {% endif %}
```

```
 <label>Voter Id:</label>
          <input type="text" name="voter_id" required><br><br>

          <button type="submit">Login</button>
       </form>
       <a href="{{url_for('Welcome')}}" class="button">Home page</a>
       {% endblock %}
   </body>
</html>
```

reg_success.html

```
<!DOCTYPE html>
<html>
<head>
   <title>Registration Successful</title>
   <link rel="stylesheet" href="{{ url_for('static', filename='css/style.css') }}">
   <style>
      .button {
         display: inline-block;
         padding: 12px 25px;
         margin: 10px;
         font-size: 16px;
         color: #fff;
         background-color: #2980b9;
         border: none;
         border-radius: 5px;
         text-decoration: none;
      }
      .button:hover {
         background-color: #1f6391;
      }
   </style>
</head>
<body>
    {% extends 'base.html' %}

   {% block title %}User Registration{% endblock %}

   {% block content %}
   <h2>Registration Successful!</h2>
   <p>Your Unique Voter ID is: <strong>{{ voter_id }}</strong></p>
   <p>Please keep this ID safe. It will be used for login and voting.</p>
   <a href="{{url_for('login')}}" class="button">Go to login</a>
   <a href="{{url_for('Welcome')}}" class="button">Home page</a>
   {% endblock %}
</body>
</html
```

result.html

```
<!DOCTYPE html>
<html>
  <head>
    <title>Results</title>
    <link rel="stylesheet" href="{{ url_for('static', filename='css/style.css') }}">
    <style>
      .button {
      display: inline-block;
      padding: 12px 25px;
      margin: 10px;
      font-size: 16px;
      color: #fff;
      background-color: #2980b9;
      border: none;
      border-radius: 5px;
      text-decoration: none;
    }
    .button:hover {
      background-color: #1f6391;
    }
    </style>
  </head>
  <body>
    {% extends 'base.html' %}

    {% block title %}User Registration{% endblock %}

    {% block content %}
    <h2>Voting Results</h2>
    <table>
      <thead>
        <tr>
          <td>Candidate name</td>
          <td>Party</td>
          <td>Vote count</td>
        </tr>
      </thead>
      <tbody>
        {% for c in r_data %}
          <tr>
            <td>{{c['name']}}</td>
            <td>{{c['party']}}</td>
            <td>{{c['vote_count']}}</td>
          </tr>
        {% endfor %}
      </tbody>
    </table>
    <a href="{{url_for('Welcome')}}" class="button">Home page</a>
    {% endblock %}
  </body>
</html>
```

vote_success.html

```
<!DOCTYPE html>
<html>
   <head>
      <title>Vote success</title>
      <link rel="stylesheet" href="{{ url_for('static', filename='css/style.css') }}">
      <style>
         .button {
         display: inline-block;
         padding: 12px 25px;
         margin: 10px;
         font-size: 16px;
         color: #fff;
         background-color: #2980b9;
         border: none;
         border-radius: 5px;
         text-decoration: none;
      }
      .button:hover {
         background-color: #1f6391;
      }
      </style>
   </head>
   <body>
      {% extends 'base.html' %}

      {% block title %}User Registration{% endblock %}

      {% block content %}
         <h1>Thanks for voting</h1>
         <h2>Your vote has been recorded successfully!!</h2>
         <a href="{{url_for('Welcome')}}">Back to Home</a>
         <a href="{{url_for('result')}}" class="button">Result</a>
      {% endblock %}
   </body>
</html>
```

voting_ballot.html

```
<!DOCTYPE html>>
<html>
   <head>
      <title>Voting Ballot</title>
      <link rel="stylesheet" href="{{ url_for('static', filename='css/style.css') }}">
      <style>
         .button {
         display: inline-block;
         padding: 12px 25px;
         margin: 10px;
         font-size: 16px;
         color: #fff;
         background-color: #2980b9;
```

```
border: none;
            border-radius: 5px;
            text-decoration: none;
        }
        .button:hover {
            background-color: #1f6391;
        }
        </style>
    </head>
    <body>
        {% extends 'base.html' %}

        {% block title %}User Registration{% endblock %}

        {% block content %}
        <h3> Vote for your candidate </h3>
        {% with messages = get_flashed_messages(with_categories=true) %}
            {% if messages %}
                {% for category, message in messages %}
                    <div class="flash {{ category }}">{{ message }}</div>
                {% endfor %}
            {% endif %}
        {% endwith %}

        <form method="POST">
            <table>
                <th>Select</th>
                <th>Name</th>
                <th>Party</th>
                <th>Description</th>
                <th>Image</th>
                {% for c in candidates %}
                    <tr>
                        <td>
                            <input type="radio" name="candidate" value="{{c['id']}}"></td>



                        <td>
                            {{ c['name'] }}
                        </td>
                        <td>
                            {{ c['party'] }}
                        </td>
                        <td>
                            {{ c['description'] }}
                        </td>
                        <td>
                            {% if c['image'] %}
```

```
                        <img src="{{ url_for('static', filename='images/' ~ c['image']) }}"
width="100">
                    {% else %}
                      No image
                    {% endif %}
                  </td>
                </tr>
              {% endfor %}
            </table>
            <button type="submit">Submit Vote</button>
        </form>
        <a href="{{url_for('Welcome')}}" class="button">Home page</a>
        {% endblock %}
    </body>
</html>
```

Welcome.html

```
<!DOCTYPE html>
<html>
    <head>
        <meta charset="UTF-8">
        <title>Welcome to E-voting System</title>
        <style>
            body {
            font-family: Arial, sans-serif;
            margin: 0;
            background-color: #f4f4f4;
            text-align: center;
        }
        .header-image {
            width: 100%;
            max-height: 400px;
            object-fit: cover;
        }
        .container {
            padding: 40px;
        }
        h1,h3 {
            color: #2c3e50;
        }
        p {
            font-size: 1.2em;
            margin: 20px 0;
        }
        .button {
            display: inline-block;
            padding: 12px 25px;
            margin: 10px;
            font-size: 16px;
            color: #fff;
            background-color: #2980b9;
            border: none;
            border-radius: 5px;
```

```
 text-decoration: none;
        }
        .button:hover {
           background-color: #1f6391;
        }
        </style>
    </head>
    <body>
        <h1>Welcome to E-voting System!!</h1>
        <h3>RSA Encryption</h3>
        <p>Secure.Transparent,Easy to use</p>

        <img src="{{url_for('static',filename='images/Evoting.png')}}"
        alt="Welcome banner"
        style="width: 100%; max-height: 500px; object-fit: contain; display: block; margin: 0 auto;">
        Not registered?<a href="{{ url_for('form') }}" class="button">Register</a>
        If already registered?<a href="{{ url_for('login') }}" class="button">Login</a>
</body></html>
```

List of Figures

Sl No.	Figure Name	Figure Title	Page No.
1.	ECC Key Generation Metrix	Figure 1.1	19
2.	RSA Key Generation Metrix	Figure 1.2	20
3.	ECC Key Generation	Figure 1.3	23
4.	ECC Signature Generation	Figure 1.4	23
5.	ECC Encryption	Figure 1.5	24
6.	ECC Decryption	Figure 1.6	24
7.	RSA Key Generation	Figure 1.7	26
8.	RSA Encryption	Figure 1.8	26
9.	RSA Decryption	Figure 1.9	27
10.	Home Page(RSA)	Figure 1.10	31
11.	Home Page(ECC)	Figure 1.11	32
12.	Registration Page (ECC)	Figure 1.12	33
13.	Login Page(ECC)	Figure 1.13	33

Abbreviation

AES	Advanced Encryption System
AES-GCM	Advanced Encryption Standard in Galois/Counter Mode
CSS	Cascading Style Sheets
DES	Data Encryption System
DICOM	Digital Imaging and Communications in Medicine
DSA	Data Structure Algorithm
ECC	Elliptic Curve Cryptography
ECDH	Elliptic Curve Diffie-Hellman
ECDLP	Elliptic Curve Discrete Logarithm Problem
EMR	Electronic Medical Record
HIPAA	Health Insurance Portability and Accountability Act
HITECH	Health Information Technology for Economic and Clinical Health Act
HKFD	Hashed message authentication code-based extract-and-expand Key Derivation Function
HTML	HyperText Markup Language
HF	Hashed Function
IFP	Interactive Flat Platform
IOT	Internet of Things
IPFS	InterPlanetary File System
PACS	Picture Archiving and Communication System
PKC	Public Key Cryptography
PoW	Proof of Work
RSA	Rivest Shamir Adleman
SKC	Secret Key Cryptography

Bibliography

[1] Stuart Harber and W. Scott Stornetta. 1991. How to Time-Stamp a Digital Document. Journal of Cryptology 1991 International Association for Cryptologic Research, 3:99-111.
[2] Satoshi Nakamoto. 2008. Bitcoin: A Peer-to-Peer Electronic Cash System.
[3] Simanta Shekhar Sarmah. 2018. Understanding Blockchain Technology. Computer Science and Engineering, 8(2): 23-29.
[4] Nentawe Y. Goshwe. 2013. Data Encryption and Decryption Using RSA Algorithm in a Network Environment. IJCSNS International Journal of Computer Science and Network Security, VOL.13 No.7.
[5] Vivek S K*, Yashank R S, Yashas Prashanth, Yashas N, Namratha M. 2020. E-Voting Systems using Blockchain: An Exploratory Literature Survey. Proceedings of the Second International Conference on Inventive Research in Computing Applications (ICIRCA-2020).
[6] Ali Kaan Koç, Emre Yavuz, Umut Can Çabuk, Gökhan Dalk l ç. Towards Secure E-Voting Using Ethereum Blockchain. Computer Engineering Dept. Dokuz Eylul University, Izmir/Turkey, Electrical-Electronics Engineering Dept. Erzincan University, Erzincan/Turkey.
[7] Gousia Habib, Sparsh Sharma, Sara Ibrahim, Imtiaz Ahmed, Shaima Qureshi and Malik Ishfaq. 2022. Blockchain Technology: Benefits, Challenges, Applications, and Integration of Blockchain Technology with Cloud Computing.
[8] Md. Navid Bin Anwar1, Mahmud Hasan2, Md. Mahade Hasan3, Jafrin Zafar Loren4 and S.M.Tanjim Hossain5. 2019. Comparative Study of Cryptography Algorithms and Its' Applications. International Journal of Computer Networks and Communications Security VOL. 7, NO. 5.
[9] Dindayal Mahto, Dilip Kumar Yadav.2017. RSA and ECC: A Comparative Analysis. International Journal of Applied Engineering Research ISSN 0973-4562 Volume 12, Number 19.
[10] Sheping Zhai1,Yuanyuan Yang, Jing Li1 , Cheng Qiu1 and Jiangming Zhao. 2019. Research on the Application of Cryptography on the Blockchain. IOP Conf. Series: Journal of Physics: Conf. Series 1168 (2019) 032077.
[11] Morgan P. McBee, ChadWilcox. 2019. Blockchain Technology: Principles and Applications in Medical Imaging. Journal of Digital Imaging.
[12] Maria-Victoria Vladucu, Ziqian Dong, Jorge Medina, Roberto Rojas-Cessa. 2023. E-Voting Meets Blockchain: A Survey. IEEE Access (Volume: 11).
[13] Ansah Jeelani Zargar, Mehreen Manzoor, Taha Mukhtar. 2017. ENCRYPTION/DECRYPTION USING ELLIPTICAL CURVE CRYPTOGRAPHY. International Journal of Advanced Research in Computer Science Volume 8, No. 7.